



2024 - 2025

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY : Data Science

Knowledge-Driven SAP Monitoring:

Graph-Enhanced Alerting and Conversational Support

By: *Chayma RHALEM*

Academic supervisor: *Hedil SAIDANI*

Corporate Internship Supervisor: *Belhassen ETTEIBI*

Avaxia

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous /
I validate the submission of the student's report:

- Nom & Prénom /Name & Surname : Chayma Rhaïem

Encadrant Entreprise/ Business site Supervisor

- Nom & Prénom /Name & Surname : Mr. Belhassen ETTEIBI

Cachet & Signature / Stamp & Signature

Sté AVAXIA
B.A 01 Imm. Montplaisir
1073 Tunis
MF : 1101801/W/A/M/000
Tél : 71 904 013

Encadrant Académique/Academic Supervisor

- Nom & Prénom /Name & Surname : Miss Hadil SAIDANI

Signature / Signature

SAIDANI Hadil



Ce formulaire doit être rempli, signé et scanné/This form must be completed, signed and scanned.

I wish to offer my deepest thanks to all those who helped me throughout this project:

*To my family,
who have been my unwavering foundation,*

*To the people who challenged me, pushed me,
and reminded me that discomfort is just growth in disguise,*

*To my professional supervisors Mr. AmenAllah Ben Salem and Mr. Belhassen Etteibi,
for patiently enduring my GPU complaints and that one time I may or may not have fried a
company computer.*

*And to that one Professor,
who's responsible for getting me into this data science rabbit hole and continues to remind me
why I walked down it.*

ACKNOWLEDGMENTS

I would like to express my deepest gratitude to **Avaxia Group** for entrusting me with the opportunity to contribute to their project. Their support, visionary guidance, and collaborative spirit have been instrumental in shaping this amazing experience.

I am immensely grateful to my professional supervisors **Mr.Belhassen ETTEIBI**, **Mr.AmenAllah BEN SALEM** and **Mr.Mohamed ABDELMOULA** for their encouragements and efforts in making this project successful. This work wouldn't have seen the light of day without their mentoring.

I particularly want to express my gratitude to Miss **Hedil SAIDANI** for her dedication and efforts in making this project successful.

Last but not least, I would like to thank the members of the jury for putting in the effort to read and evaluate my work. Your feedback is very important to me.

CONTENTS

Contents	iii
List of Figures	vi
List of Tables	viii
1 General Introduction	2
2 Project Context and Overview	5
2.1 Project Scope	5
2.1.1 Hosting Organisation	6
2.1.2 Organizational Structure	7
2.2 Problem Definition	7
2.3 Existing Solutions and Competitors	8
2.4 Methodology	11
3 Theoretical Background	17
3.1 Fundamentals of Artificial Intelligence	17
3.2 Large Language Models (LLMs)	21
3.3 Graph-Based Knowledge Representation	22
3.4 Retrieval-Augmented Generation (RAG)	23
3.4.1 RAG Architecture and Workflow	23
3.4.2 Dense Retrieval Methods	24
3.5 Fine-Tuning	25
3.6 RAG vs. Fine-tuning	26
3.7 Chatbot Fundamentals	27
4 Business Understanding	28
4.1 State-of-the-Art	28
4.1.1 What is an ERP	28
4.1.2 SAP BASIS and Infrastructure consultant	29
4.1.3 What are T-codes	29

5	Data Understanding and acquisition	31
5.1	Data Sources	31
5.2	Transaction Code Examples and Data Structure Analysis	33
5.2.1	Transaction Code Data Examples	34
5.2.2	Data Structure Variations and Challenges	36
5.3	Metadata-Aware Scraping and Content Extraction	37
5.3.1	Implementation Strategy for Nested Documents	37
5.4	Data Exploration and Anomaly Definition	38
5.5	Knowledge Base Construction and Data Organization	39
5.5.1	Knowledge Base Development Process	39
5.5.2	Chunking Strategy	40
5.5.3	Enhanced Chunking Strategy Implementation	40
5.5.4	Adaptive Chunking Strategy for Knowledge Graph Construction	41
5.5.5	Special Content Handling	41
5.6	Hierarchical Chunking Strategy for Vector Store	42
5.6.1	Document Structure Preservation	43
5.6.2	Multi-Level Content Processing	43
5.6.3	Chunk Size Optimization	43
5.6.4	Quality Assurance Measures	43
5.6.5	Comparative Analysis: Adaptive vs. Hierarchical Chunking	44
6	Modeling and Evaluation	46
6.1	Model Architecture and Pipeline	46
6.2	Knowledge Graph Construction and Management	48
6.2.1	Graph Database Technologies	48
6.2.2	Graph Neural Networks for Knowledge Graphs	49
6.2.3	Knowledge Graph Embeddings	49
6.2.4	Multi-Strategy Triplet Extraction Pipeline	51
6.2.5	Component Selection and Evaluation	52
6.2.6	Relation Extraction Models	55
6.3	Cross-Chunk Inference Strategy Analysis	57
6.3.1	Composite Pipeline Design: Multi-Strategy Triplet Extraction	57
6.3.2	SAP Notes Knowledge Graph Construction	59
6.4	Graph Evaluation: LLM-as-a-Judge with G-Eval	60
6.4.1	Motivation for LLM-as-a-Judge	60
6.4.2	Evaluation Framework: G-Eval with Chain-of-Thought Reasoning	60
6.4.3	Advantages Over Other Methods	63
6.5	Hybrid RAG Approach	64
6.5.1	Vector-First Graph-Enriched RAG Architecture	64
6.5.2	Multi-Stage Retrieval Pipeline	65
6.5.3	Knowledge Graph Integration	66
6.5.4	Advanced SAP-Specific Capabilities	66

6.5.5	Prompt Engineering Strategy	66
6.5.6	Evaluation Strategy	67
6.5.7	Retrieval Architecture	67
6.5.8	Vector Database Selection Justification	68
6.5.9	Embedding Model (EmbeddingModel)	71
6.5.10	Vector Database (VectorDatabase)	72
6.5.11	Beam Search	72
6.5.12	G-Eval Assessment Framework	74
6.6	Chatbot Implementation	76
7	Deployment	79
7.1	Integration with SAP Data Sources	79
7.2	Backend Infrastructure	80
7.3	Frontend Interface	80
7.4	Chatbot and Evaluation Integration	81
7.5	Authentication and Security	81
7.5.1	JWT-Based Authentication	81
7.5.2	Session Management	82
7.6	Caching and Performance Optimization	82
7.6.1	Multi-Level Caching Strategy	82
7.6.2	Cache Invalidation	82
7.7	LLM Inference Engines and Integration	82
7.7.1	Ollama (Local Inference)	82
7.7.2	GROQ (Cloud Inference)	82
7.7.3	Model Selection Justification	83
7.8	Deployment Configuration	84
7.9	User Interface and Monitoring Workflows	84
7.9.1	Use Case Diagram	85
7.9.2	Authentication and Access Control	86
7.9.3	Alert Notification System	91
7.9.4	Interactive Chatbot Interface	92
7.9.5	Core Features Implementation	92
7.9.6	User Interface Components	93
7.9.7	Performance Optimization	94
8	Conclusion and Perspectives	96

LIST OF FIGURES

1.1	Gartner Hype Cycle for Artificial Intelligence, 2024	3
2.1	Avaxia Group Logo [1]	6
2.2	"CRISP-DM framework" Lifecycle phases [2]	12
2.3	An Outline of the Steps of the KDD Process [3]	14
2.4	"Microsoft TDSP" Methodology lifecycle [5]	15
3.1	Hierarchical relationship between AI, ML, DL, and Generative AI. ¹	18
3.2	Transformer architecture Adapted from "Attention is All You Need". ² . .	20
3.3	Basic RAG Pipeline[6]	23
4.1	SAP's Logo ³	28
5.1	Scrapy's Logo ⁴	32
5.2	Playwright's logo ⁵	33
5.3	Data ingestion Pipeline	37
5.4	Web Scraping Architecture and Flow	38
5.5	Adaptive Chunking Algorithm Flow	42
6.1	Pygeometric's Logo[8]	50
6.2	Training loss curve showing rapid convergence with early stopping at epoch 7	51
6.3	Examples from the SAP Help documents graph	59
6.4	Examples from the SAP Notes graph	60
6.5	Triplet Evaluation Scores using G-Eval on SAP Documents	63
6.6	The process of beam search (beam size=2) ⁶	73
6.7	Layered graph of HNSW ⁷	74
6.8	G-Eval Performance Analysis showing metric distributions, correlations, and overall system performance trends	75
6.9	High-level architecture of the SAP Basis Assistant chatbot	77
7.1	VMware Horizon Client logo[10]	79
7.2	End-to-end Deployment Architecture	80
7.3	Technology stack for serving RESTful APIs	80

7.4	Frontend technologies used in the web interface	81
7.5	Ollama's logo[15]	83
7.6	Groq's Logo[16]	83
7.7	Complete System Interaction Flow - Authentication, Monitoring, Chat, and Alert Sequences	85
7.8	System Use Case Diagram	86
7.9	View of login and registration interfaces	87
7.10	Systems Dashboard with Real-time Health Status	87
7.11	system-wide alert with traceability context	88
7.12	Transaction Code Monitoring Output Example	89
7.13	Transaction Code Context from databases	90
7.14	Example of MS Teams Notification for SAP Anomalies	91
7.15	Chatbot Interaction Examples	93
7.16	User Profile and Model Configuration Page	95

LIST OF TABLES

2.1	Benchmarking of monitoring tools for SAP-centric environments	9
2.2	Stakeholder analysis and success metrics	11
2.3	Comparison of CRISP-DM, TDSP, and KDD methodologies	16
3.1	Comparison of Transformer Architecture Variants	21
3.2	Comparison of Pertinent Embedding Models for Dense Retrieval	25
3.3	Comparison between RAG and Fine-tuning Mialon et al., 2023	26
6.1	LLaMA Model Variants for RAG Tasks - Technical Specifications and Hardware Requirements	47
6.2	Comparison of graph database technologies	48
6.3	Common Knowledge Graph Embedding Models	50
6.4	Limitations of Single NLP Models and the Need for a Pipeline	52
6.5	Coreference Resolution Models Comparison for SAP Technical Documen- tation	53
6.6	SAP Acronym Format Variations and Handling Strategies	53
6.7	SAP Table Types and Corresponding Processing Methods	54
6.8	Table-to-Text Model Evaluation for SAP Content	54
6.9	Named Entity Recognition Models for SAP Technical Content	55
6.10	Knowledge Graph Quality Assessment Framework	59
6.11	Comparison of Modeling Approaches for SAP Monitoring	64
6.12	Comprehensive Vector Database Evaluation Matrix	69
6.13	Qdrant Configuration Optimized for SAP Workloads	71
6.14	Qdrant Configuration for SAP Monitoring System	72
6.15	Used Language Models and Specifications	78
7.1	Comparison between Ollama and GROQ inference engines	83

LIST OF ACRONYMS

ERP	Enterprise Resource Planning
SAP	Systems, Applications, and Products in Data Processing
T-code	Transaction Code
RAG	Retrieval-Augmented Generation
NLP	Natural Language Processing
AI	Artificial Intelligence
ML	Machine Learning
NER	Named Entity Recognition
RE	Relation Extraction
EL	Entity Linking
GNN	Graph Neural Networks
GCN	Graph Convolutional Networks
KGE	Knowledge Graph Embeddings
TPLM	Table Pre-trained Language Models
ASGI	Asynchronous Server Gateway Interface
CSV	Comma-Separated Values
JSON	JavaScript Object Notation
API	Application Programming Interface
FAISS	Facebook AI Similarity Search
KB	Knowledge Base
KPI	Key Performance Indicator
LLM	Large Language Model
RCA	Root Cause Analysis
BLEU	Bilingual Evaluation Understudy metric
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
METEOR	Metric for Evaluation of Translation with Explicit Ordering
MS Teams	Microsoft Teams
SDG	Sustainable Development Goals

GENERAL INTRODUCTION

SAP systems are the foundation of paramount business operations in many different industries in modern enterprise landscape. These Enterprise Resource Planning (ERP) systems manage critical processes in areas like supply chain, logistics, customer relations, human resources, and financial operations.

Effectively maintaining SAP environments is a major challenge for companies relying on this software, especially when it comes to knowledge management across distributed implementations, transaction analysis, and system monitoring.

The complexity of SAP environments is notable due to enormous volumes of transactions, the substantial corpus of documentations and the dispersed nature of the applications in different departments.

Both Business administrators and Clients often struggle to:

- Extract actionable insights from the transactions logs that are found across multiple systems with different architectures.
- Identify the relevant SAP Notes and helpful documentation relevant to specific system behaviors
- Proactively detect potential system failures and issues or if there is space for a voluntary optimization opportunity.
- Efficiently assist consultants with the extensive knowledge base required for SAP Systems administration.

Motivation and Industry Context

The integration of artificial intelligence (AI) into enterprise systems represents a significant frontier in the evolution of business technologies. According to the Gartner Hype Cycle for Artificial Intelligence 2024¹), several key technologies that are relevant and at the core of this project are positioned at strategic points along the innovation curve.

¹ Gartner Inc., Hype Cycle for Artificial Intelligence 2024. Available at: <https://www.gartner.com/en/articles/hype-cycle-for-artificial-intelligence>

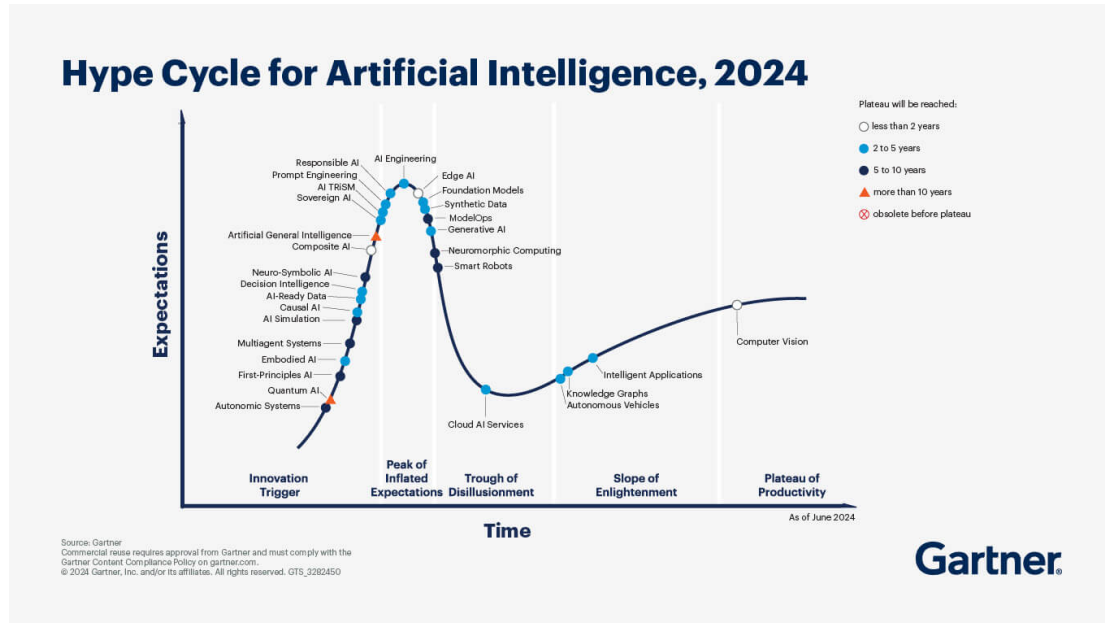


Figure 1.1: *Gartner Hype Cycle for Artificial Intelligence, 2024*

Knowledge Graphs, the central component of our approach, are currently traversing the "Slope of Enlightenment", which demonstrates their maturity potential toward practical business value after passing through earlier hype cycles. This positioning is proof that knowledge graphs have overcome their initial inflated expectations and disillusionment, with organizations now developing realistic understanding of its applications and benefits in enterprise contexts.

Simultaneously, Generative AI approaches appear near the "Peak of Inflated Expectations," reflecting the intense interest and sometimes unrealistic expectations surrounding these technologies. Our project acknowledges this positioning by implementing a hybrid approach that leverages the strengths of generative AI while constraining it through structured knowledge representation to overcome its limitations.

The Placement of Intelligent Applications on the Slope of Enlightenment reinforces our approach of building a specialized enterprise application that combines multiple AI technologies to address the aforementioned SAP systems concerns. By integrating these technologies at different maturity stages, this project tries to produce tangible business value, while also overcoming the Hype Cycle's inherent adoption obstacles.

Recent progress made in the fields of **Artificial intelligence (AI)** and **natural language processing (NLP)**, especially through large language models (LLM), have opened the door to endless possibilities to build intelligent assistants and monitoring agents within an enterprise's ecosystems. LLMs offer strong capabilities in language understanding and generation, yet they suffer from inherent limitations including restricted context windows, hallucination of irrelevant material, and difficulties reasoning over structured or tabular data.

To overcome these limitations, the paradigm of **Retrieval-Augmented Generation (RAG)** has emerged, allowing LLMs to access external sources of information during

inference. Although RAG systems typically rely on vector-based semantic search over unstructured documents, they often struggle with deep structural understanding of relationships between entities, which limits the precision and traceability of generated responses in sensitive business uses.

To address these shortcomings, recent research has proposed the use of **Graph-Enhanced RAG** approaches, which integrate representative knowledge (via knowledge graphs) into the generative process. By enriching RAG architectures with structured domain representations, it becomes possible to improve both the factual consistency and the interpretability of generated alerts or recommendations.

Project Objectives

The main objective of this graduation project is to effectively design and implement a hybrid RAG system for intelligent alerting and monitoring of SAP transactional activity.

This implementation combines technology at various phases of the Gartner Hype Cycle to strike a balance between innovation and practical application. By disciplining generative AI capabilities within a structured framework of knowledge graphs, the project hopes to contribute to the creation of a universal enterprise-ready system administration solution that avoids the frequent pitfalls of emergent technologies while maximizing on their strengths.

The initiative also seeks to address major issues in monitoring, such as anomaly detection, knowledge grounding, and user-centric explainability. The rest of this report discusses the theoretical foundations, design decisions, implementation details, and experimental assessments of an intelligent monitoring agent for SAP systems.

PROJECT CONTEXT AND OVERVIEW

This chapter will concentrate on providing the necessary background information so that the remaining chapters may be smoothly followed. We will mainly concentrate on the ideas that are most crucial to us in the context of this project as there are many details to consider.

2.1 Project Scope

Enterprise resource planning (ERP) solutions, like SAP, are crucial for managing important business activities in today's data-driven economy. Any irregularities or interruptions in transaction processing have the potential to cause serious operational inefficiencies, monetary losses, and erode client confidence. Thus, preserving the accuracy and dependability of SAP transactions requires the implementation of a strong anomaly detection system.

Traditional monitoring methodologies, predominantly reliant on predefined heuristic rules, exhibit substantial limitations in their capacity to detect complex anomalies within the multidimensional parameter space of SAP transactions. These conventional approaches frequently generate excessive false positives due to their inability to incorporate contextual understanding and relationship patterns inherent in transaction data structures. This graduation project addresses these limitations through the development of an intelligent anomaly detection framework powered by Graph-enhanced Retrieval Augmented Generation (RAG) technology, specifically engineered to identify, contextualize, and explain anomalies in SAP transaction processing environments.

Business Objectives

The primary objectives of this project initiative encompass:

1. **Enhancing Transaction Monitoring:** Development of a monitoring system that continuously surveils SAP transactions for anomalies.
2. **Optimization of Incident Response Protocols:** Implementation of automated alert generation mechanisms enriched with traceable and contextually enriched

notifications, thereby minimizing Mean Time To Resolution (MTTR) for transaction-related incidents.

3. Improving Decision-Making: Provide actionable insights through detailed alerts that help stakeholders make informed decisions regarding transaction management and troubleshooting.
4. Increasing operational efficiency: Streamline operations by reducing manual oversight and enabling proactive management of transaction-related anomalies.
5. Mitigation of Language Model Limitations: Addressing fundamental challenges associated with Large Language Models (LLMs) in professional settings, especially hallucinations, context window constraints, and structured data interpretation shortages through an innovative and robust pipeline architecture.

2.1.1 Hosting Organisation

AVAXIA GROUP is a multinational information technology consulting firm founded in 2006, with strategic presence across multiple regions including headquarters in Dubai and operational centers in Tunisia, Japan, and Canada. The organization specializes in enterprise technology solutions with particular emphasis on Enterprise Resource Planning (ERP) implementation, systems integration, and managed services for mission-critical business applications. Their technical expertise extends across the SAP ecosystem and complementary platforms including Salesforce, ensuring optimal implementation outcomes and system performance.



Figure 2.1: Avaxia Group Logo [1]

Corporate Philosophy and Approach

Avaxia Group adopts a distinctly customer-centric methodology in its service delivery model, emphasizing collaborative partnerships rather than transactional relationships.

Service Portfolio

Avaxia offers a comprehensive portfolio of digital and consulting services, with a strong focus on enterprise solutions, infrastructure support, and emerging technologies. Its SAP services provide full-scale infrastructure management and 24/7 proactive support, delivering tailored solutions that align with clients' strategic objectives. In parallel, Avaxia specializes in modern software development and deployment, employing recent technologies to support both cloud and on-premise integration.

Avaxia Group is structured around two specialized departments, each offering distinct service domains:

- **Functional Consultancy:** Business solution design for the new deployment of SAP functional modules, analysis and optimization of end-to-end processes, management of functional deployment and test planning.
- **Big Data:** Services including data management, transformation, integration, storage, and analysis. Their talents cover data pipeline development, ETL processes, and data governance.
- **VR / AR:** Development of interactive VR environments, enabling users to navigate, interact with objects, and engage with other users in real-time. Applications include monitoring systems, simulations, training programs, and virtual tours.

2.1.2 Organizational Structure

The organizational architecture of Avaxia Group is structured to facilitate specialized expertise while maintaining cross-functional collaboration. The SAP department, where this research is conducted, employs a hierarchical structure with functional specialization:

- **Divisional Organization** This holds the responsibilities for each department/-division based on the function and role to deal with business development and strategic decisions.
- **Operational Organization** globally focuses on the technical challenges and expertise driving projects and services (plan, execution, delivery, and support) It also serves to track, evaluate and optimize projects/support operations and streamline technical processes at a regular basis and encompassing various activities within each team.
- **Corporate Structure** Supports the front office representing the internal operations (Finance, HR, Training, Marketing, Service quality, IT), to ensure the continuity of the business as a whole and to provide a methodical environment.

Our project unfolds within the Quadient team, situated in the SAP Infrastructure and BASIS department at Avaxia Group's Tunisia office. The responsibilities of this team encompass monitoring and support for mission-critical SAP systems across multiple client engagements, making it an ideal environment for developing and validating an advanced RAG-based anomaly detection system. The cross-functional nature of transaction monitoring requires collaboration with multiple stakeholders in the organizational hierarchy, including infrastructure specialists, database administrators and technical consultants.

2.2 Problem Definition

The current landscape of SAP system monitoring presents several fundamental challenges:

Manual Monitoring Challenges: Current monitoring processes are largely manual, leading to delays in identifying anomalies and increased risk of oversight.

Data Overload: The sheer volume of transaction data makes it challenging for teams to effectively monitor and analyze for anomalies without automated systems.

Inconsistent Anomaly Detection: Existing methods for detecting anomalies may lack accuracy, leading to false positives or negatives that hinder effective response efforts.

Lack of Contextual Insights: Alerts generated from existing systems often lack sufficient context, making it difficult for users to understand the nature of the anomaly or how to address it.

Knowledge Management Challenges SAP expertise is specialized, it Requires years of experience to develop basic proficiency in SAP and it's also fragmented, Knowledge exists in large documentations or personal experience.. More importantly, there are always New SAP versions, patches, and configurations that constantly change the landscape

2.3 Existing Solutions and Competitors

Current Monitoring Landscape

Most organizations currently rely on:

- Standard SAP monitoring tools: Built-in transaction codes for system monitoring
- Third-party monitoring platforms: Generic IT monitoring solutions adapted for SAP
- Manual procedures: Regular checks performed by BASIS consultants
- Rule-based alerting: Predefined thresholds that trigger notifications

Table 2.1: *Benchmarking of monitoring tools for SAP-centric environments*

Tool	Type	Pricing	Pros	Cons
SAP Solution Manager (Sol-Man)	On-premises	Included with SAP license	Native SAP support, integrated t_code monitoring	Steep learning curve, lacks semantic or AI features
Splunk	Cloud / On-prem	Paid	Advanced log analytics, flexible dashboarding	SAP integration requires customization, no semantic reasoning
Elastic Stack (ELK)	Open-source	Free / Paid tiers	Real-time logs, Kibana dashboards, scalable ingestion	Requires custom SAP integration, lacks domain logic
Microsoft Azure Monitor	Cloud-based	Paid	Rich cloud monitoring, built-in anomaly detection	Requires SAP-specific agents and adapters
SAP Leonardo ML	SAP Cloud Suite	Paid	SAP-native ML services, real-time ERP integration	Limited flexibility, cloud-only, can be very expensive for big systems
Zabbix	Open-source	Free	Lightweight, highly customizable, SNMP support	No native SAP awareness, lacks semantic/log correlation
Datadog	Cloud-based	Paid	Modern UI, strong cloud/service integration	Expensive at scale, limited SAP t_code observability
AppDynamics	Cloud-based	Paid	APM with business transaction tracing, Cisco-backed	Requires complex setup, SAP integration effort needed
Instana	Cloud-based	Paid	Real-time observability, automated root cause analysis	Limited flexibility, not SAP transaction-aware
Proposed AI-Based Solution	Hybrid	Open-source	SAP-specific anomaly detection, semantic search, intelligent alerts, cross-KB reasoning	Requires infrastructure setup, still emerging tech stack

Competitive Advantage

These existing solutions highlight the competitive landscape within which our project operates. However these solutions often require:

- Extensive configuration for SAP-specific use cases
- Significant manual interpretation of alerts

- Limited contextual understanding of SAP business processes
- High false positive rates that overwhelm technical teams

Project Motivation

By focusing specifically on SAP transactions while leveraging cutting-edge generative AI techniques that are affordable, our solution aims to fill gaps left by these broader platforms by providing tailored insights directly relevant to the client and the SAP consultants. The integration of open source Large Language Models (LLMs) adds a sophisticated layer of contextual understanding, setting our system apart with enhanced precision, cost-effectiveness and relevance in anomaly detection within SAP environments.

Proposed Monitoring Solution Overview

The proposed solution introduces a modern, AI-driven monitoring assistant tailored specifically for SAP systems. Unlike traditional monitoring tools that rely on fixed rules or manual thresholds, this assistant uses a hybrid retrieval approach that combines flexibility with structure, allowing it to explain system issues, suggest resolutions, and prioritize alerts intelligently.

The system is designed to:

- Understand a wide range of SAP-related issues, even when phrased differently (semantic search)
- Interpret relationships between system components and transactions (structured graph logic)
- Reduce irrelevant or incorrect alerts by grounding responses in verified documentation

This hybrid approach ensures both **broad coverage** and **high precision**, critical to fast and accurate SAP Basis decision-making.

Key Components of the solution

- **Context-Aware Alerting:** The system continuously monitors SAP transactions and detects issues in near real-time. Alerts are generated with severity, root cause analysis, and recommended actions, helping teams respond faster and more effectively.
- **Smart Document Retrieval:** The assistant searches across SAP documentation, notes, and alert definitions using modern AI techniques. Instead of relying solely on keywords and words similarity, it understands meaning, allowing it to retrieve relevant information even from complex or technical documents.
- **Knowledge-Based Reasoning:** SAP content is structured into a knowledge graph that maps relationships between transactions, errors, systems, and solutions. This

helps the assistant explain not just what happened, but why and how it's connected to other events by grounding outputs in structured facts.

- **Centralized API and Logic Layer:** A centralized backend manages alert rules, query flows, and data processing. It ensures all parts of the system—from detection to recommendation—work seamlessly together.
- **Modern User Interface and Team Integration:** A clean, web-based interface gives teams access to real-time alerts, system health summaries, and on-demand Tcode insights. The system is also connected to Microsoft Teams to notify users when high-priority issues occur.

The table below will further clarify how each component of this solution will be considered by the stakeholders.

Table 2.2: *Stakeholder analysis and success metrics*

Stakeholder	Primary Needs	Success Metrics
SAP BASIS Consultants	Real-time anomaly visibility; Contextual alerting for t_codes; Operational prioritization	Reduced MTTR; Improved first-time resolution; Decreased manual effort
Client Organizations	Minimal disruption to SAP operations; Transparent monitoring; Compliance assurance	Fewer critical failures; Higher availability; Visibility into system health
IT Operations Teams	Actionable alerts; Seamless incident workflow; Prioritized signal over noise	Reduced false positives; Higher operator productivity; Accurate root cause correlation
Executive Management	High-level operational KPIs; Risk and ROI tracking; Alignment with digital strategy	Reduced operational risk; Resource optimization; Business impact visibility

These criteria acknowledge the complexity of enterprise system monitoring and the limitations of current AI tools. The outlined targets represent realistic and measurable goals for assessing the technical and operational success of the project in its various phases.

2.4 Methodology

The selection of an appropriate methodology for data science and AI engineering projects requires careful consideration of the unique characteristics that differentiate these projects from traditional software development:

Why Traditional Agile/Scrum May Not Be Optimal

Uncertainty and Exploration: Data science projects involve significant uncertainty about the quality and viability of the data available, model performance, and feasibility of

approaches. Unlike traditional software development, where requirements are relatively stable, data science projects require extensive exploration and experimentation phases that don't fit well within fixed sprint cycles.

Non-linear Progress:

Traditional software development follows a relatively predictable path from requirements to implementation. Data science projects often require iterative hypothesis testing, where negative results (e.g., a model that doesn't perform well) are valuable learning experiences rather than failures, making traditional velocity metrics less meaningful.

Research Component: Research activities don't align well with the predictable deliverables expected in Agile sprints.

CRISP-DM

CRISP-DM, which stands for Cross-Industry Standard Process for Data Mining, is an industry-proven way to guide a data mining project.

It includes descriptions of the typical phases of a project, the tasks involved with each phase, and an explanation of the relationships between these tasks. The life cycle model consists of six phases with arrows indicating the most important and frequent dependencies between phases. The sequence of the phases is not strict. In fact, most projects move between phases as necessary.

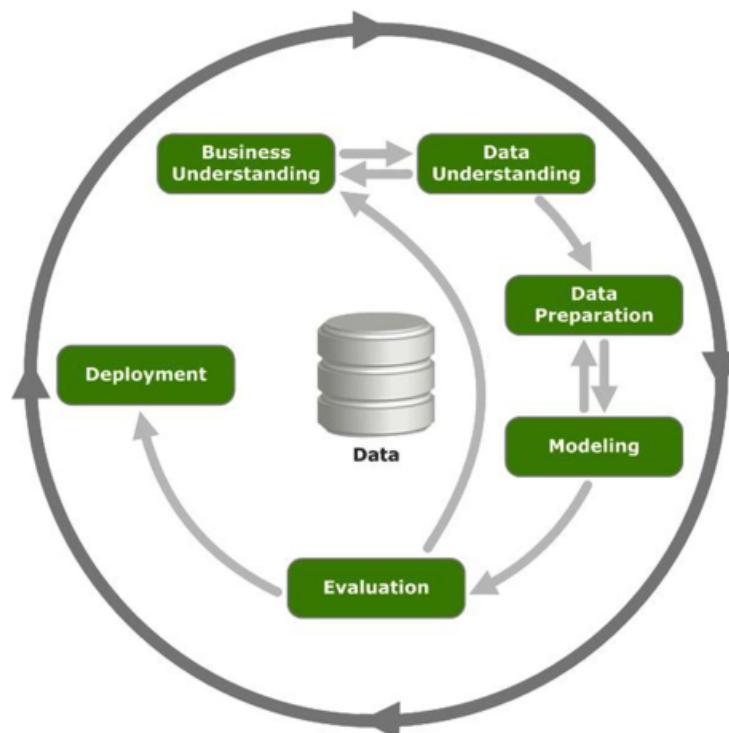


Figure 2.2: "CRISP-DM framework" Lifecycle phases [2]

- **Business Understanding:** Define project objectives and translate business goals into data mining problems
- **Data Understanding:** Collect initial data, assess quality, and discover preliminary insights
- **Data Preparation:** Clean, transform, and format data for modeling requirements
- **Modeling:** Select and apply modeling techniques with optimized parameters
- **Evaluation:** Assess model results against business objectives and success criteria
- **Deployment:** Implement findings in production and establish monitoring strategy

Advantages:

- **Business-Driven:** Starts with business understanding, aligning perfectly with enterprise needs such as SAP anomaly interpretation and impact analysis.
- **Iterative Workflow:** Encourages continuous feedback and model refinement, which is useful for improving alert accuracy and relevance.
- **Technology-Agnostic:** CRISP-DM doesn't impose tooling constraints, is well suited for flexible integration with modern stacks.
- **Widely Adopted:** Industry-proven and supported by rich documentation and tooling, making communication easier.
- **Focus on Evaluation:** Includes phases that are consecrated to evaluating both the technical performance and business adequacy essential for SAP monitoring applications.

Limitations:

Lacks Built-in MLOps: Does not natively address modern deployment practices (CI/CD, monitoring, versioning) found in TDSP or modern pipelines.

Knowledge Discovery in Databases (KDD)

The Knowledge Discovery in Databases (KDD) process, introduced by Fayyad et al. in 1996, represents one of the earliest formal methodologies for extracting actionable insights from large volumes of data. Unlike methodologies that focus mainly on the modeling or deployment phase, KDD emphasizes the entire end-to-end lifecycle from data preprocessing to knowledge interpretation.

The main objective of KDD is to uncover valid, useful, and understandable patterns in data that can be used for insights and interpretations.

KDD Process Steps

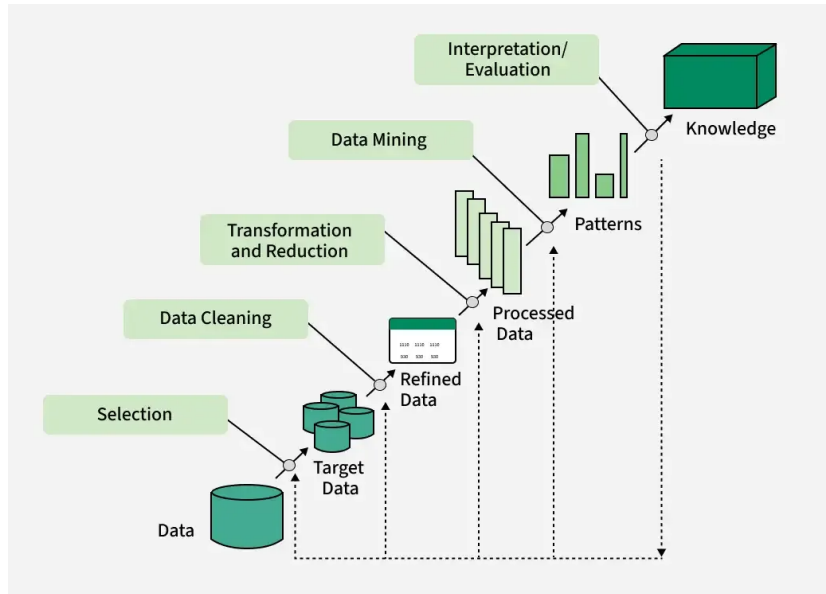


Figure 2.3: *An Outline of the Steps of the KDD Process [3]*

Advantages

- Strong emphasis on knowledge validity, novelty, and usefulness.
- Promotes a complete and iterative data analysis pipeline.

Disadvantages

- Limited guidance on deployment or operationalization.
- Less emphasis on agile practices or software engineering principles.

TDSP: Team Data Science Process

The Team Data Science Process is a framework developed by Microsoft to guide data science projects. It consists of several stages, each addressing different aspects of the data science lifecycle. TDSP's project lifecycle is like CRISP-DM and includes five iterative stages:

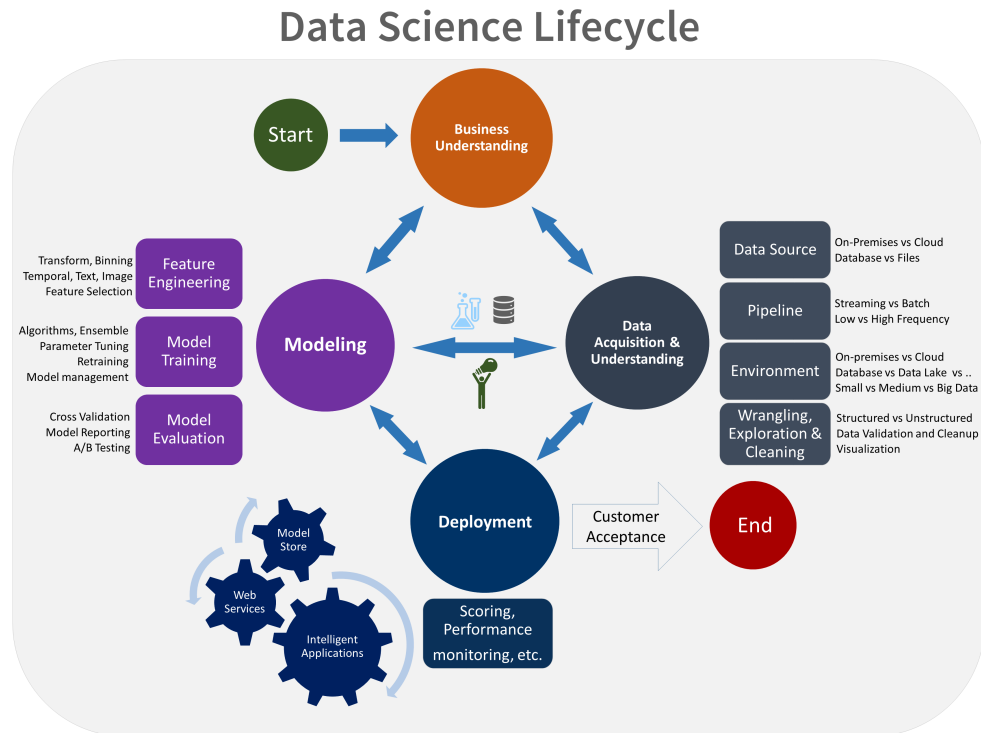


Figure 2.4: "Microsoft TDSP" Methodology lifecycle [5]

1. Business Understanding: define objectives and identify data sources
2. Data Acquisition and Understanding: ingest data and determine if it can answer the presenting question (effectively combines Data Understanding and Data Cleaning from CRISP-DM)
3. Modeling: feature engineering and model training (combines Modeling and Evaluation)
4. Deployment: deploy into a production environment

Advantages

- Agile: Emphasizes the need for incremental deliverables.
- Familiar: The product backlog, features, user stories, bugs, Git versioning, and sprint planning are familiar to those used to common software practices.
- Data Science Native: TDSP acknowledges that data science and software engineering are different, and is built for data science teams working on production-bound projects.
- Flexible: TDSP can be implemented as it is defined or in conjunction with other approaches such as CRISP-DM.

Disadvantages

- Fixed Sprints: TDSP leverages fixed-length planning sprints which many data scientists struggle with.

- Some Inconsistencies: Not all of Microsoft’s documentation is consistent.

Comparison of Methodologies

Table 2.3: Comparison of CRISP-DM, TDSP, and KDD methodologies

Aspect	CRISP-DM	TDSP	KDD
Main Focus	Structured, iterative process for business-driven data mining	Agile lifecycle for scalable ML projects	Extracting patterns and knowledge from raw data
Business Integration	Strong: Business understanding is the starting point	Strong: Built-in stakeholder alignment phase	Limited: Focused more on data and patterns
Data Preparation	Extensive and iterative	Strong, often automated	Central phase
Modeling Guidance	Supported, model selection and tuning included	Strong support with tracking	Present but less structured
Deployment Support	Moderate: Deployment included, but less prescriptive	Strong: Emphasizes production-readiness and DevOps	Minimal
Human Interpretability	Emphasized during evaluation and business understanding	Moderate: Focused on performance and reproducibility	High: Patterns must be meaningful to humans
Pattern Evaluation	Integrated into evaluation and business context validation	Via performance metrics and testing pipelines	Core focus: Central to methodology
Best Use Case	Data mining and analytics with strong business context	Iterative, collaborative machine learning workflows	Discovering actionable insights from raw, unlabeled data

Chosen Methodology

CRISP-DM was adopted as the primary project methodology due to its business alignment, technical neutrality, and emphasis on iterative improvement, a key requirement for building a usable and adaptable SAP monitoring assistant.

Conclusion

This chapter underscores the strategic importance of developing a SAP anomaly detection system that addresses critical gaps in current monitoring approaches within the CRISP-DM methodology, the project aims to transform reactive SAP monitoring into proactive, context-aware anomaly detection.

THEORETICAL BACKGROUND

This chapter will focus on giving the reader the right background knowledge to easily follow through the rest of the chapters. Due to the enormous amount of details one can get in, we will only focus on the concepts that are most important to us in the context of this project.

3.1 Fundamentals of Artificial Intelligence

This section will briefly take us through the difference between Artificial intelligence, Machine Learning, Deep Learning and Generative AI. While these terms are usually used interchangeably, they do hold differences. Therefore, let's start with some definitions.

Artificial Intelligence (AI)

Artificial Intelligence is a subset of the field of Computer Science that focuses on making computers intelligent. It started from the 60s with machines that try to converse like humans and do tasks that were meant for humans. The early stages of Artificial Intelligence were lists of rules and logic implemented by developers in big systems that would later be called expert systems.

Machine Learning (ML)

Machine Learning is a subset of artificial intelligence, where the machine has to learn rules from the data itself without being explicitly told and programmed to do so by the developers. Machine Learning programs are based on mathematical equations that can learn mappings from given inputs to given outputs. This allows them to find clues that humans may miss and thus get better results at whatever tasks they are trained on.

Deep Learning (DL)

Deep learning is a subfield of machine learning that analyzes data and discovers intricate patterns by using multi-layered artificial neural networks. Because of its inspiration from the structure and operation of the human brain, computers are able to process

information similarly. Natural language processing, image and speech recognition, and many other AI applications now rely heavily on deep learning as a key technology.

Key Concepts

Artificial Neural Networks (ANNs): Deep learning models are built upon ANNs, which are interconnected nodes (neurons) which are the foundation of deep learning models.

Deep learning uses multiple layers (often hundreds or thousands) within the neural network, in contrast to traditional machine learning. As a result, the network can learn more intricate data representations. **Feature Extraction:** Manual feature engineering is no longer necessary because deep learning models can automatically extract pertinent features from raw data. "Learning from Data:"

Learning from Data: Large datasets are used to teach deep learning algorithms, which then modify their internal parameters to perform better over time.

Generative AI

Generative AI encompasses models designed to produce new content including text, images, audio, and other data modalities. These systems learn complex statistical patterns from large training corpora and utilize this knowledge to generate human-like content. Key capabilities include:

- Content generation across multiple modalities
- Pattern recognition and synthesis
- Contextual understanding and adaptation
- Creative and analytical task performance

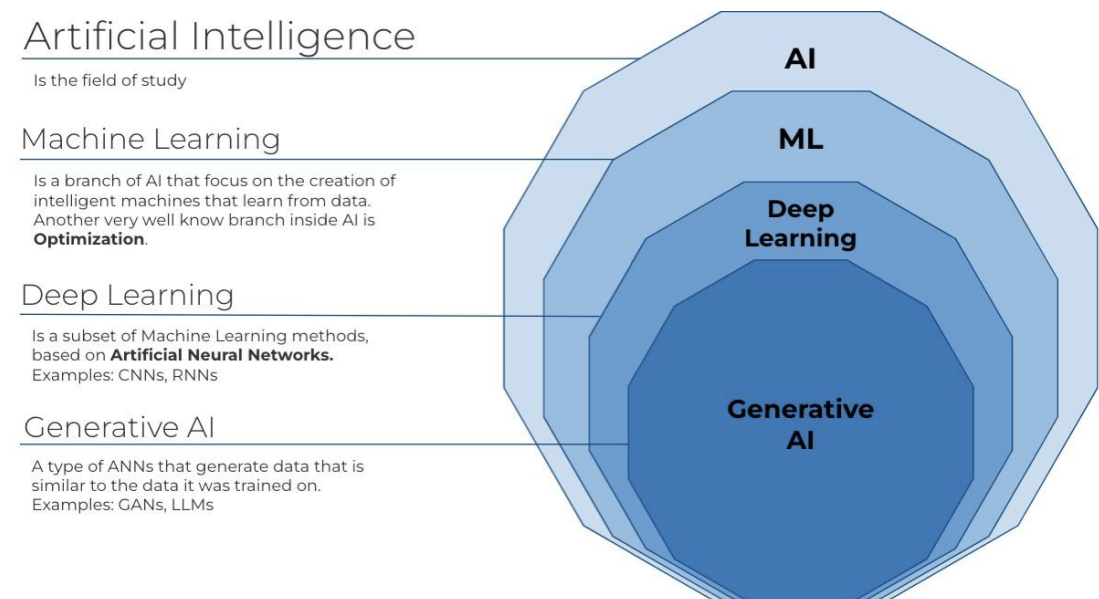


Figure 3.1: Hierarchical relationship between AI, ML, DL, and Generative AI. ¹

¹ <https://www.devoteam.com/expert-view/unlimited-creativity-how-generative-ai-is-transforming-the-world-of-innovation/>

Natural Language Processing (NLP)

Natural Language Processing represents a multidisciplinary field combining computer science, linguistics, and artificial intelligence to enable computational understanding and manipulation of human language. NLP systems must address the inherent complexity and ambiguity of natural language across diverse contexts and domains.

Key components of NLP used in this project include:

- **Tokenization:** Segmentation of text into discrete linguistic units
- **Named Entity Recognition (NER):** Identification and classification of specific entities within text
- **Part-of-Speech Tagging:** Grammatical categorization of words and phrases
- **Dependency Parsing:** Analysis of syntactic relationships between words
- **Semantic Analysis:** Extraction of meaning and intent from textual content
- **Discourse Analysis:** Understanding of text structure and coherence
- **Relation Extraction:** To construct relationships between extracted entities and support the formation of a Knowledge Graph.
- **Text Summarization and Classification:** To reduce document size and classify incoming SAP messages or logs by category or severity.

Evolution from Classical to Neural Approaches

The field of NLP has undergone significant paradigmatic shifts, transitioning from rule-based and statistical methods to neural architectures.

Classical NLP Methods

Traditional approaches relied on:

- **Rule-based Systems:** Hand-crafted linguistic rules and grammar specifications
- **Statistical Methods:** N-gram models, Hidden Markov Models, and probabilistic frameworks
- **Feature Engineering:** Manual extraction of relevant linguistic features
- **Bag-of-Words Models:** Simplified text representation ignoring word order
- **TF-IDF Weighting:** Term frequency-inverse document frequency for relevance scoring

Neural NLP Revolution

The advent of deep learning transformed NLP through:

- **Word Embeddings:** Dense vector representations capturing semantic relationships
- **Recurrent Neural Networks:** Sequential processing with memory capabilities
- **Convolutional Neural Networks:** Local pattern detection in text
- **Attention Mechanisms:** Dynamic focus on relevant input portions

- **Transfer Learning:** Pre-training on large corpora followed by task-specific fine-tuning

The Transformer Revolution

The introduction of the Transformer architecture in "Attention is All You Need" [Vaswani et al., 2017](#) revolutionized NLP by:

- Eliminating recurrence in favor of self-attention mechanisms
- Enabling parallel processing of sequences
- Capturing long-range dependencies effectively
- Facilitating large-scale pre-training on massive text corpora

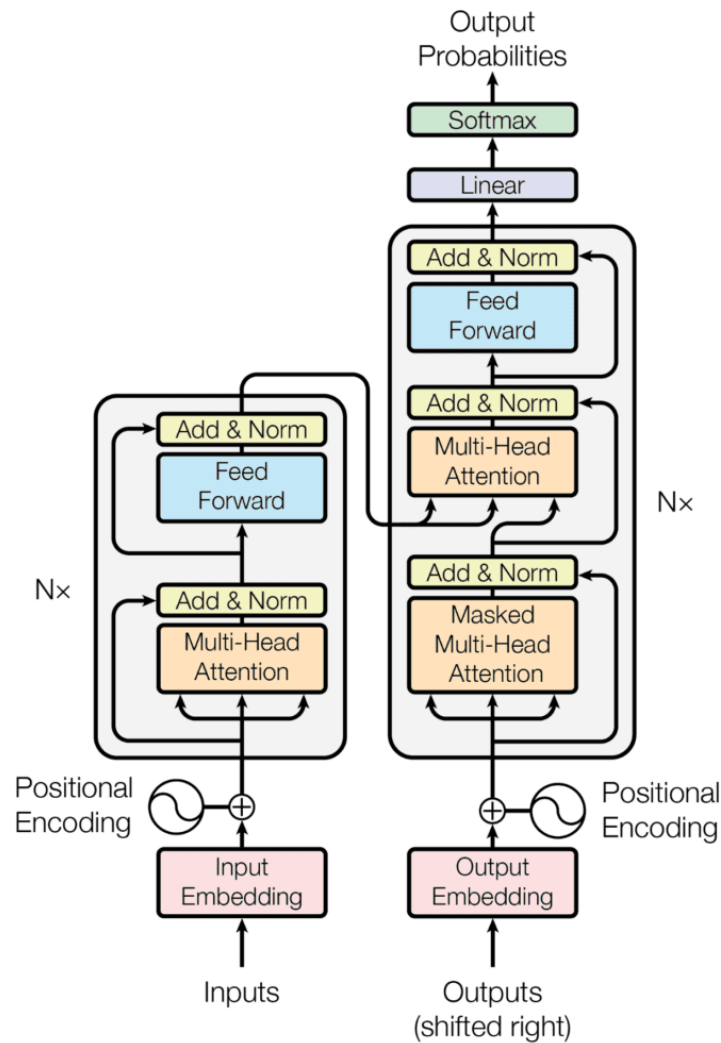


Figure 3.2: Transformer architecture Adapted from "Attention is All You Need".²

² <https://arxiv.org/abs/1706.03762>

This paradigm shift enabled more accurate, context-aware systems—especially valuable in enterprise applications like document summarization, anomaly explanation, and intelligent query handling.

3.2 Large Language Models (LLMs)

Large Language Models represent the current state-of-the-art in natural language processing, built upon transformer architectures and trained on vast textual corpora. These models demonstrate remarkable capabilities in language understanding and generation across diverse domains and tasks.

Transformer Architecture Variants

Modern LLMs can be categorized into three primary architectural paradigms, each optimized for different classes of natural language processing tasks:

Table 3.1: *Comparison of Transformer Architecture Variants*

Architecture	Components	Example Models	Primary Applications
Encoder-only	Bidirectional self-attention layers enabling full context access	BERT, RoBERTa, DistilBERT	Text classification, entity recognition, question answering, sentiment analysis
Decoder-only	Causal self-attention layers with autoregressive generation capability	GPT-3/4, LLaMA, PaLM, Claude	Text generation, completion, dialogue systems, code synthesis
Encoder-Decoder	Bidirectional encoder coupled with causal decoder via cross-attention	T5, BART, Flan-T5	Translation, summarization, sequence transformation tasks

Emergent Capabilities

As models scale, they develop qualitatively new capabilities not present in smaller variants:

- **In-Context Learning:** Ability to adapt to new tasks from examples in the input prompt
- **Chain-of-Thought Reasoning:** Step-by-step problem decomposition and logical inference
- **Few-Shot Learning:** Task performance with minimal training examples
- **Instruction Following:** Adherence to complex natural language instructions
- **Code Understanding:** Comprehension and generation of programming languages
- **Mathematical Reasoning:** Solution of complex mathematical problems

Reinforcement Learning from Human Feedback (RLHF)

RLHF aligns model outputs with human preferences through a multi-stage training process:

1. **Supervised Fine-tuning:** Initial adaptation on high-quality human demonstrations
2. **Reward Model Training:** Learning a reward function from human preference comparisons
3. **PPO Optimization:** Policy optimization using the learned reward model

3.3 Graph-Based Knowledge Representation

Knowledge Graphs: Foundations and Structure

Knowledge graphs provide structured representation of real-world entities and their interconnections through a graph-theoretic framework. Unlike traditional relational databases, knowledge graphs offer flexible schema evolution and natural representation of complex, multi-hop relationships.

Graph Theoretical Foundations

A knowledge graph can be formally defined as a directed labeled graph $G = (V, E, L)$ where:

- V represents the set of vertices (entities)
- $E \subseteq V \times V$ represents the set of edges (relationships)
- L is a labeling function that assigns semantic labels to edges

Knowledge graphs typically follow the Resource Description Framework (RDF) triple format:

$$\text{Triple} = (\text{Subject}, \text{Predicate}, \text{Object}) \quad (3.1)$$

Entity Resolution and Linking

Entity resolution addresses the challenge of identifying when different mentions refer to the same real-world entity:

- **String Similarity Metrics:** Levenshtein distance, Jaccard similarity, and fuzzy matching
- **Embedding-Based Matching:** Dense vector representations for semantic similarity
- **Graph-Based Resolution:** Leveraging existing relationships for disambiguation
- **Machine Learning Approaches:** Supervised and unsupervised entity linking models

3.4 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation represents a paradigm that enhances language model capabilities by incorporating external knowledge sources during generation. This approach addresses key limitations of parametric models while maintaining their generative flexibility.

Motivation and Theoretical Foundations

Standard language models encode knowledge within their parameters during pre-training, leading to several limitations:

- **Knowledge Cutoff:** Information is frozen at training time
- **Hallucination:** Generation of plausible but factually incorrect content
- **Attribution:** Difficulty tracing generated content to sources
- **Specialization:** Limited domain-specific knowledge depth

RAG addresses these limitations by augmenting the generation process with relevant external information retrieved at inference time.

3.4.1 RAG Architecture and Workflow

The RAG pipeline consists of three primary components operating in sequence:

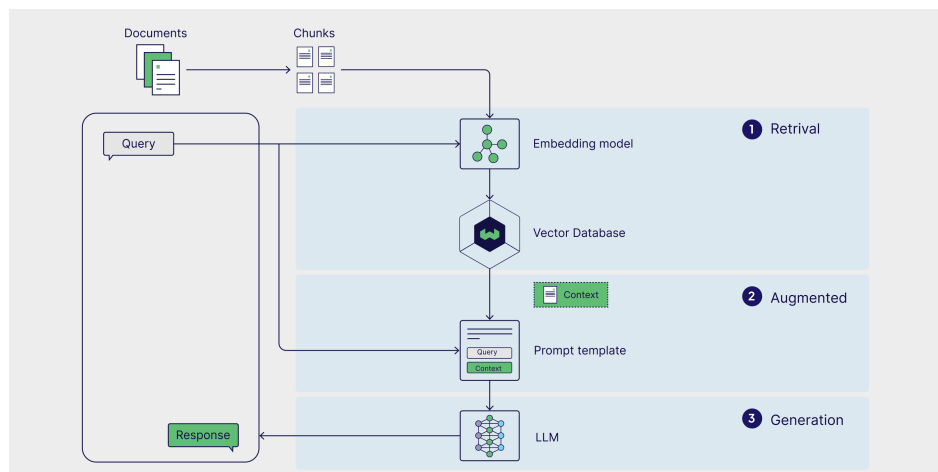


Figure 3.3: Basic RAG Pipeline[6]

Knowledge Encoding Phase

External documents are processed and stored in a searchable format:

1. **Document Chunking:** Segmentation of documents into semantically coherent segments.
2. **Embedding Generation:** Conversion of text chunks to dense vector representations using encoder models

3. **Index Construction:** Storage of embeddings in vector databases optimized for similarity search

Document Chunking Methods:

- **Fixed-size chunking:** Equal character/token segments (simple but may break semantic units)
- **Semantic chunking:** Paragraph or sentence boundaries (preserves meaning)
- **Recursive chunking:** Hierarchical splitting with overlap (maintains context)
- **Hierarchy / Metadata-aware chunking:** Section/header-based splitting (preserves document structure)

Chunking Parameters:

- **Chunk size:** Typically 64–2048 tokens (context vs. precision trade-off)
- **Overlap:** 10–30% overlap between chunks (maintains continuity)
- **Boundary detection:** Sentence/paragraph delimiters

Retrieval Phase

Given a query, relevant knowledge is retrieved from the index:

$$\text{Retrieved} = \text{TopK}(\text{similarity}(E(q), E(d_i))) \quad (3.2)$$

where $E(\cdot)$ represents the embedding function, q is the query, and d_i are document chunks.

Generation Phase

The language model generates responses conditioned on both the query and retrieved context:

$$P(\text{response} \mid \text{query}) = P(\text{response} \mid \text{query}, \text{retrieved_docs}) \quad (3.3)$$

3.4.2 Dense Retrieval Methods

Modern RAG systems employ dense retrieval techniques that represent queries and documents in high-dimensional semantic vector spaces.

Embedding Models

Various embedding models offer different trade-offs between accuracy, efficiency, and specialization:

Similarity Metrics

Common similarity measures for dense retrieval include: **Cosine Similarity:**

$$\text{cos_sim}(a, b) = \frac{a \cdot b}{|a| |b|} \quad (3.4)$$

Table 3.2: Comparison of Pertinent Embedding Models for Dense Retrieval

Model	Dimensions	Max Tokens	Training Method	Performance
text-embedding-ada-002	1536	8191	Contrastive	High
all-MiniLM-L6-v2	384	256	Sentence-BERT	Medium
all-mpnet-base-v2	768	384	Sentence-BERT	High
E5-large-v2	1024	512	Contrastive	Very High
BGE-large-en	1024	512	Contrastive	Very High
nomic-embed-text-v1	768	8192	Contrastive	Very High

Euclidean Distance:

$$\text{euclidean}(a, b) = \sqrt{\sum_{i=1}^d (a_i - b_i)^2} \quad (3.5)$$

Dot Product: Direct computation of $a \cdot b$ for normalized vectors

3.5 Fine-Tuning

Fine-tuning adapts pre-trained LLMs to specific tasks or domains by continuing their training on curated datasets. The process involves taking a foundational model and exposing it to additional training examples that demonstrate the desired behavior, responses, or knowledge for a particular use case.

During fine-tuning, the model’s parameters are adjusted through additional training iterations, allowing it to learn patterns specific to the target domain while retaining its general language capabilities.

The approach offers significant benefits for specialized applications, enabling models to produce more accurate, contextually appropriate responses within specific domains. Several methodological approaches are commonly employed:

Full fine-tuning: Traditional approach that updates all model parameters during training, providing maximum adaptability but requiring substantial computational resources and memory.

Parameter-Efficient Fine-Tuning (PEFT): Modern techniques that achieve comparable performance while updating only a small subset of parameters, significantly reducing computational costs and memory requirements.

Low-Rank Adaptation (LoRA): A popular PEFT method that introduces trainable low-rank matrices alongside frozen pre-trained weights, allowing efficient adaptation with minimal parameter overhead.

Adapter methods: Insert small trainable modules between existing layers while keeping the original model frozen, enabling task-specific customization with reduced resource requirements.

3.6 RAG vs. Fine-tuning

The table below compares RAG vs. fine-tuning.

Table 3.3: Comparison between RAG and Fine-tuning *Mialon et al., 2023*

Feature Comparison	RAG	Fine-tuning
Knowledge Updates	Directly updates the retrieval knowledge base, ensuring information remains current without the need for frequent retraining.	Stores static data, requiring re-training for knowledge and data updates.
External Knowledge	Proficient in utilizing external resources, particularly suitable for documents or other structured/unstructured databases .	Can be applied to align the externally learned knowledge from pretraining with large language models, but may be less practical for frequently changing sources
Data Processing	Requires minimal data processing and handling .	Relies on constructing high-quality datasets .
Model Customization	Focuses on information retrieval and integrating external knowledge but may not fully customize model behavior or writing style .	Allows adjustments of LLM behavior, writing style, or specific domain knowledge based on specific tones or terms.
Interpretability	Answers can be traced back to specific data sources , providing higher interpretability and traceability.	Like a black box , not always clear why the model reacts a certain way, with relatively lower interpretability.
Computational Resources	Requires computational resources to support retrieval strategies and technologies related to databases . External data source integration and updates need to be maintained.	Preparation and curation of high-quality training datasets , definition of fine-tuning objectives, and provision of corresponding computational resources are necessary.
Latency Requirements	Involves data retrieval, potentially leading to higher latency .	LLM after fine-tuning can respond without retrieval, resulting in lower latency .
Reducing Hallucinations	Inherently less prone to hallucinations as each answer is grounded in retrieved evidence.	Can help reduce hallucinations by training the model based on specific domain data but may still exhibit hallucinations when faced with unfamiliar input.

To summarize the table: RAG offers Large Language Models the direct access to factual and up-to-date information. This integration allows LLMs to fetch precise and verified facts directly from relevant databases and knowledge repositories in real-time. While fine-tuning can address some of these aspects by adapting the model to specific data, RAG resolves the issue of knowledge limit without the substantial costs associated with fine-tuning. Additionally, RAG's approach to leveraging external sources is flexible and scalable, allowing for easy updates and adjustments without the need for extensive retraining. Fine-tuning adapts the style, tone, and vocabulary of LLMs so that your linguistic "paint brush" matches the desired domain and style. RAG does not provide this level of customization in terms of linguistic style and vocabulary. However, a successful LLM application typically involves connecting specialized data to the LLM workflow. Once you have a functional application, you can add fine-tuning to enhance the style and vocabulary of the system.

3.7 Chatbot Fundamentals

A chatbot is an AI-powered conversational agent designed to interact with users through natural language processing. In the context of SAP monitoring, our chatbot serves as an intelligent interface that can analyze transaction anomalies, interpret system alerts, and provide contextual guidance for resolution.

Types of Chatbots

Modern chatbots can be categorized into several types based on their underlying architecture and capabilities:

Rule-Based Chatbots: Follow predefined conversation flows and decision trees. Limited in handling complex or unexpected queries.

Retrieval-Based Chatbots: Select responses from a predefined corpus based on input matching. More flexible than rule-based systems but constrained by existing response databases.

Generative Chatbots: Generate responses dynamically using language models. Offer greater flexibility and contextual understanding.

Hybrid AI-Optimized Chatbots: Combine retrieval mechanisms with generative capabilities, enhanced by knowledge graphs and vector databases for improved accuracy and context awareness.

Conclusion

This chapter provides the essential theoretical background to understand the technologies used in this project. By combining foundational concepts in artificial intelligence with advanced retrieval and generation techniques, we are laying the foundation of a robust framework for building an intelligent, knowledge-driven systems capable of scalable and context-aware decision assistance.

BUSINESS UNDERSTANDING

The Business Understanding chapter outlines the context and objectives of the anomaly detection project within the framework of the CRISP-DM. This chapter emphasizes the importance of aligning technical efforts with business goals, ensuring that the developed solution addresses real-world challenges faced by stakeholders.

4.1 State-of-the-Art

4.1.1 What is an ERP

Enterprise Resource Planning (ERP) is a software system that integrates and manages key business processes like finance, human resources, manufacturing, supply chain, sales, and procurement. It provides a single source of truth for an organization, eliminating data duplication and silos and improving operational efficiency. ERP solutions help businesses gain real-time visibility into operations, automate processes, and make data-driven decisions, ultimately enhancing overall performance and agility.

What is SAP

SAP, short for Systems, Applications and Products in Data Processing, is a leading provider of enterprise application software. It's known for its Enterprise Resource Planning (ERP) software, which helps businesses manage various aspects of their operations, including finance, human resources, supply chain, and customer relationship management.



Figure 4.1: SAP's Logo ¹

SAP software suite Modules examples:

¹ Source: <https://www.sap.com/>

- **SAP EWM** (Extended Warehouse Management): SAP EWM is a specialized module that focuses on warehouse optimization and inventory management processes. It assists companies in managing their warehouse operations efficiently and ensuring that the right products are available at the right time.
- **SAP ECC** (Enterprise Central Component): SAP ECC is central to the ERP module of SAP. It includes various modules such as finance, sales, procurement, manufacturing, and human resources. ECC provides end-to-end business process integration, allowing companies to manage their operations within a unified platform.
- **SAP Solution Manager** (SOLMAN): SAP Solution Manager is a central management and monitoring tool provided by SAP, helps in application lifecycle management, project management, IT service management, and business processes.

4.1.2 SAP BASIS and Infrastructure consultant

A SAP Basis and Infrastructure consultant is charged with the technical infrastructure of an SAP environment like installations, upgrades, and application patches. This resource is responsible for resolving complications such as connectivity issues and also providing support component adjustments or external system and database assistance. The role of an SAP Basis Infrastructure consultant requires the following tasks:

1. System Administration
2. Performance Monitoring and Tuning
3. Security Administration
4. Database Administration
5. Technical Troubleshooting
6. Backup and Recovery
7. Integration and Interface Management
8. System Upgrades and Migrations
9. Documentation and Training

4.1.3 What are T-codes

In SAP, Transaction Codes (T-codes) are short alphanumeric commands used to quickly access specific functions, reports, or system diagnostics. They serve as entry points into SAP's functional modules, allowing users to bypass deep menu structures and directly execute tasks such as job monitoring, queue management, or performance analysis.

Each T-code corresponds to a backend function or program—typically implemented in ABAP, that performs a defined set of operations. When a T-code is executed, SAP checks user permissions, runs the associated logic, and presents the user with an interactive interface (form, report, or status screen).

In the context of SAP system administration and Basis operations, T-codes are essential for monitoring the health, stability, and performance of different system

components. They provide real-time insight into logs, queues, job statuses, dumps, background processing, and more.

Use of T-codes in This Project

This project focuses on proactive monitoring using a selected set of high-relevance T-codes that provide comprehensive coverage of SAP system behavior:

- **ST22** – Analyze ABAP runtime errors (short dumps).
- **DB02** – Used to analyze and monitor database statistics (db growth, table space size, missing index & etc).
- **SM37** – Monitor background jobs.
- **SP01** – Check spool (print) requests and statuses.
- **SM66** – View global work process overview.
- **SM58** – Analyze transactional RFC errors (asynchronous processing).
- **SMQ1, SMQ2** – Monitor outbound and inbound queues in qRFC/tRFC.
- **SM12, SM13** – Review locked entries and update records
- **ST03N** – Analyze workload and performance statistics.
- **SOST** – Monitor SAPconnect messages (emails, faxes, etc.)
- **ST13** – Run Basis tools for deeper diagnostics and logs.

These T-codes were selected based on their criticality for real-time operations, relevance to Basis teams, and frequency of use during incident resolution. By continuously monitoring these transactions and interpreting their outputs through AI-powered reasoning, the system enables early detection of anomalies and supports faster root cause analysis.

Conclusion

This chapter underscores the strategic importance of developing an anomaly detection system tailored for SAP transactions within the CRISP-DM Methodology. By aligning technical capabilities with business objectives while incorporating success metrics, this project aims to enhance operational efficiency, improve decision-making processes, and ensure compliance with regulatory standards.

DATA UNDERSTANDING AND ACQUISITION

The Data Acquisition and Understanding chapter is essential for establishing a robust foundation for the anomaly detection system developed for SAP transactions. This chapter details the systematic collection, processing, and analysis of data necessary to understand the business context, define alerting conditions, and identify anomalous transaction patterns. It emphasizes the importance of knowledge base (KB) documents created based on business needs, which guide the monitoring and alerting processes.

Understanding Transaction Codes in SAP

Overview of SAP Transaction Codes

In SAP, transaction codes (Tcodes) are alphanumeric identifiers that enable users to quickly access specific functions or screens within the SAP environment. Each Tcode corresponds to a particular transaction or action, such as viewing reports, entering data, or executing programs. For instance, the Tcode SM37 is used to monitor background jobs in SAP.

5.1 Data Sources

SAP Transaction Logs

These logs capture detailed records of transactions processed within the SAP system. Each entry typically includes attributes such as transaction codes (Tcode), timestamps, user IDs, error messages, and other metadata. This data is essential for identifying patterns and detecting anomalies in transaction processing.

One of the ways to get Tcodes records (e.g., ST22 in this example) is to use SAP GUI Navigation to access ABAP runtime errors logs, a *daily routine activity* for all SAP BASIS consultants that they have to repeat manually multiple times during their shifts.

Real-time Data Extraction

For real-time extraction of transaction codes and their associated data, the SAP Team engineers have implemented PyRFC (Python RFC Connector) to establish direct connections with the SAP system. PyRFC's capabilities to call SAP Remote Function Calls (RFCs) directly from Python allowed us to have a ready-for-use and scalable automated extraction of Tcode records without manual intervention through the SAP GUI.

The PyRFC implementation provides several advantages:

- **Real-time Access:** Continuous monitoring and extraction of transaction data as it occurs in the SAP system.
- **Automated Processing:** Eliminates the need for manual SAP GUI navigation that BASIS consultants typically perform multiple times per shift.
- **Structured Data Retrieval:** Direct access to SAP function modules and tables containing transaction logs and metadata.

SAP Help Portal Documentation

To build a comprehensive understanding of SAP transaction codes and their functionalities, we systematically scraped documentation from the SAP Help Portal. This external data source provides detailed technical documentation, configuration guides, and troubleshooting information for various SAP modules and transaction codes.

Web Scraping Implementation

We employed multiple web scraping technologies to efficiently extract documentation from the SAP Help Portal:

- **Scrapy:** Scrapy was chosen as the primary crawler due to its high performance, asynchronous request handling, and native support for handling large-scale structured crawling.



Figure 5.1: *Scrapy's Logo*¹

- **Playwright:** We Integrated Playwright into the pipeline to deal with the dynamic and JavaScript-heavy components of the portal that Scrapy alone could not access. We selected Playwright over Selenium for several reasons:

¹ <https://github.com/scrapy/scrapy>

² <https://playwright.dev/>



Figure 5.2: Playwright's logo ²

- *Performance*: Playwright offers faster execution due to event-driven architecture.
- **BeautifulSoup**: Used in the final parsing layer to process and clean the raw HTML collected by Scrapy and Playwright. BeautifulSoup was particularly effective in extracting nested content (e.g., tables, bullet lists, headings) and transforming it into semantically organized text.

The scraping process was designed to respect the portal's robots.txt guidelines and implement appropriate delays to avoid overwhelming the server's infrastructure.

SAP Notes and Internal Documentation

SAP Notes provide critical troubleshooting information, known issues, and configuration recommendations for specific transaction codes and system scenarios. Due to restricted access to SAP Notes through the SAP Launchpad, these documents were primarily provided by the company's SAP team, supplemented by targeted extraction where access permissions allowed.

The SAP Notes collection includes:

- Technical troubleshooting guides for specific t_codes and systems using the components and versions specified for each.
- Known error patterns and their resolutions depending on the component and patch as well.
- System configuration recommendations and symptoms diagnosis with solutions.
- Performance optimization guidelines

5.2 Transaction Code Examples and Data Structure Analysis

This section provides anonymized examples of the different data structures encountered during the data acquisition process, demonstrating the variety and complexity of information sources used in building the anomaly detection system.

5.2.1 Transaction Code Data Examples

The following examples illustrate the diverse structures of transaction data extracted through PyRFC.

SP01 - Spool Request Monitor

Transaction Purpose: Monitor and manage spool requests and print jobs within the SAP system.

Listing 5.1: *SP01 Transaction Data Structure*

```
{
  "Spool No.": "119973",
  "User Name": "USER****",
  "Date": "2024-09-09",
  "Status": "Error",
  "System": "ECC-***-02",
  "Timestamp": "2024-10-10 10:47:50.626",
  "sid": "***",
  "t_code": "SP01",
}
```

Key Monitoring Parameters:

- Error rates in spool processing
- Completion time for print job processing

SM12 - Lock Entry Display

Transaction Purpose: Display and manage lock entries to monitor system locks and potential deadlock situations.

Listing 5.2: *SM12 Transaction Data Structure*

```
{
  "Client": "620",
  "User name": "USER****",
  "Time": "11:50:22",
  "Date": "2024-12-12",
  "Table Name": "LUBU",
  "Timestamp": "2024-12-12 11:50:28.580",
  "sid": "***",
  "t_code": "SM12",
}
```

Key Monitoring Parameters:

- Duration of lock entries and their frequencies on critical tables
- User-specific locking patterns

SM37 - Background Job Monitor

Transaction Purpose: Monitor background jobs, their status, and execution details for system automation processes.

Listing 5.3: SM37 Transaction Data Structure

```
{
  "Job ": "Z3_WM_*****_UPDATE",
  "Job Created By": "*****",
  "Status ": "Cancelled",
  "Start Date ": "2025-04-24",
  "Start Time ": "19:08:01",
  "End Date ": "2025-04-24",
  "End Time ": "19:08:01",
  "Timestamp ": "2025-04-25 10:59:14.105",
  "sid ": "***",
  "t_code ": "SM37",
}
```

Key Monitoring Parameters:

- Job completion rates and failure patterns
- Execution time analysis for scheduled jobs

ST22 - ABAP Runtime Error Analysis

Transaction Purpose: Analyze ABAP runtime errors and system dumps for troubleshooting and system stability monitoring.

Listing 5.4: ST22 Transaction Data Structure

```
{
  "Current Date ": "2024-12-11",
  "Time ": "14:46:45",
  "Application Server ": "ECC-***-00_***_00",
  "User Name ": "*****",
  "Client ": "620",
  "Name of runtime error ": "DBIF_RSQL_INVALID_REQUEST",
  "Timestamp ": "2024-12-12 11:49:32",
  "sid ": "***",
  "t_code ": "ST22",
}
```

Key Monitoring Parameters:

- Frequency of specific runtime errors
- User and system patterns in error occurrence
- Application server distribution of errors

5.2.2 Data Structure Variations and Challenges

Transaction Code Variability

The analysis of transaction data reveals significant structural variations:

- **Field Diversity:** Different `t_codes` generate completely different field sets
- **Data Types:** Mixed data types including timestamps, status codes, and descriptive text
- **Naming Conventions:** Inconsistent field naming across different transaction types
- **Optional Fields:** Variable presence of fields depending on transaction context

Integration Solutions

To handle the structural diversity, we implemented:

- **Schema Mapping:** Dynamic field mapping for different transaction types
- **Metadata Preservation:** Maintaining original field names and structures
- **Type Standardization:** Consistent data type handling across sources
- **Timestamp Normalization:** Unified timestamp formats for temporal analysis

Data Cleaning and Preprocessing

Ensured data quality and consistency by:

- Removing duplicates from scraped documentation and transaction logs.
- Handling missing values in both structured transaction data and unstructured documentation.
- Normalizing data formats and timestamps across different data sources.
- Standardizing document formats from various scraping sources.
- Text preprocessing for scraped documentation including tokenization, cleaning HTML artifacts, and format standardization.

The raw transaction logs and scraped documentation were structured into formats suitable for further processing and analysis.

Data Ingestion

Implemented comprehensive data ingestion strategies to handle multiple data sources:

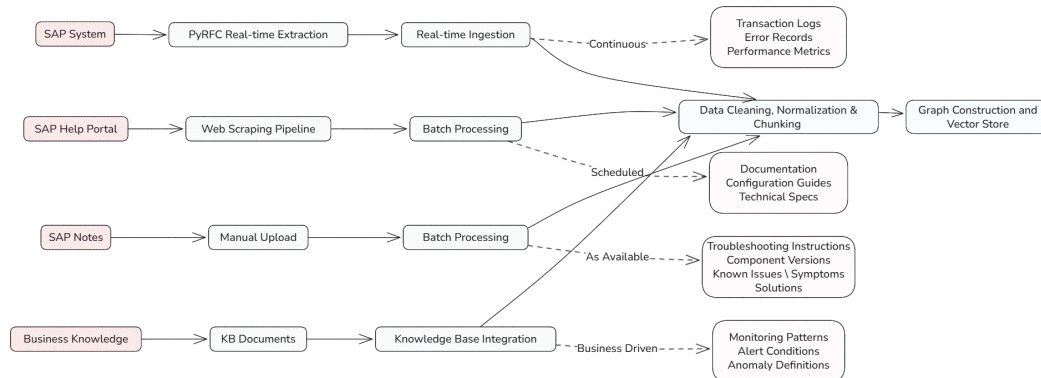


Figure 5.3: Data ingestion Pipeline

- **Real-time Ingestion:** PyRFC-based continuous extraction of transaction logs
- **Batch Processing:** Scheduled scraping of SAP Help Portal documentation
- **Manual Upload:** Integration of company-provided SAP Notes and internal documentation
- **Knowledge Base Integration:** Structured storage of business-driven knowledge base documents

This multi-source approach ensured that all relevant data was readily available for processing while maintaining data lineage and source attribution.

5.3 Metadata-Aware Scraping and Content Extraction

A very important part of the data acquisition strategy was to implement metadata-aware scraping that preserves the hierarchical structure and contextual relationships within the scraped documents to ensure that the semantic meaning and organizational structure of the original documentation are maintained.

5.3.1 Implementation Strategy for Nested Documents

The SAP Help Portal contains complex nested documentation structures that require specialized handling:

Multi-level Navigation Handling

Our scraping framework addresses the challenge of nested documents by:

- **Breadcrumb Tracking:** Maintaining navigation paths to preserve document location within the overall structure
- **Parent-Child Relationships:** Explicitly capturing relationships between main documents and their sub-documents

- **Cross-Reference Mapping:** Identifying and preserving links between related documents across different sections
- **Context Inheritance:** Ensuring that nested documents inherit relevant context from their parent documents

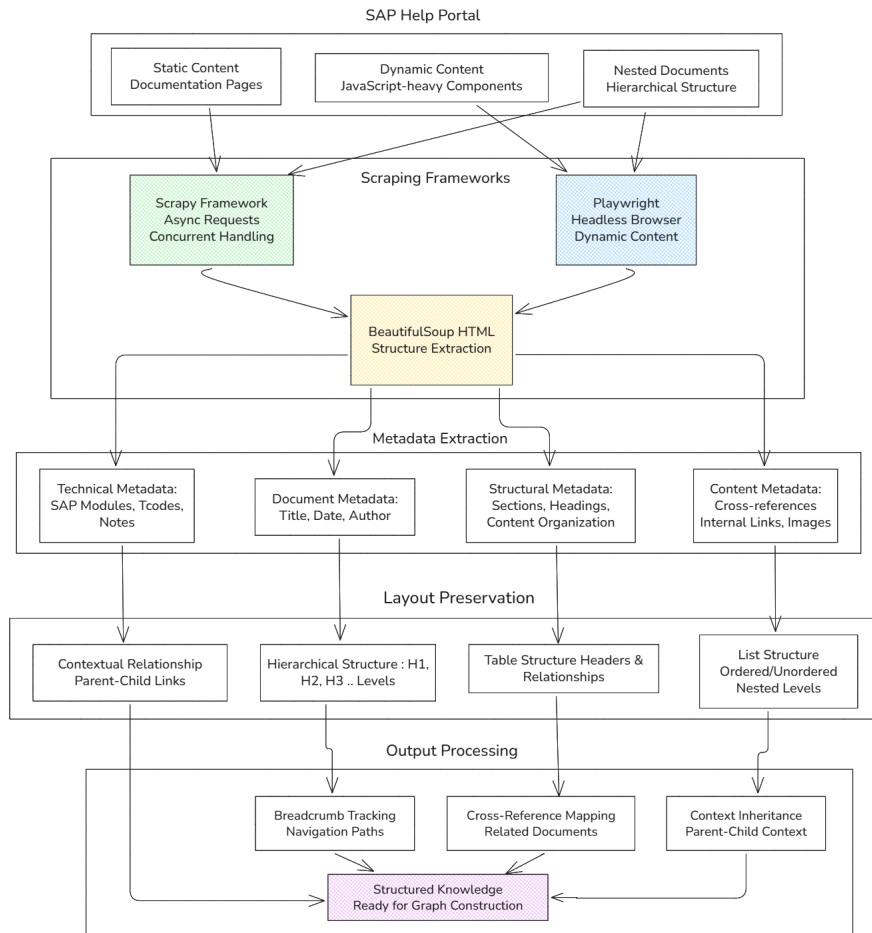


Figure 5.4: Web Scraping Architecture and Flow

5.4 Data Exploration and Anomaly Definition

We defined what constitutes an anomaly within the context of SAP transactions based on historical analysis, scraped documentation insights, and business insights gathered during meetings and discussions in the office.

Turns out anomalies in our system are not limited to individual Tcodes behaving abnormally. Instead, anomalies can manifest as:

- **Individual Transaction Anomalies:** Deviations from expected patterns such as unusual spikes in error rates or unexpected delays in processing times for specific Tcodes.
- **Transaction Pattern Anomalies:** Groups of normal Tcodes that, when appearing in specific combinations or with unusual repetition patterns, indicate potential

system issues.

- **Temporal Anomalies:** Normal transactions occurring at unusual times or frequencies that deviate from established patterns.
- **Sequential Anomalies:** Patterns of transaction sequences that, while individually normal, collectively indicate anomalous system behavior.

These patterns are identified through cross-referencing with SAP Notes and documentation, combined with historical transaction data analysis.

5.5 Knowledge Base Construction and Data Organization

The creation of the Knowledge Base (KB) was a collaborative process that involved direct engagement with SAP BASIS consultants and administrators. The KB development and retrieval architecture follows a hierarchical approach designed to optimize information retrieval for anomaly detection.

5.5.1 Knowledge Base Development Process

Step 1: Collaborative Identification of Critical Patterns

Working directly with SAP BASIS consultants and administrators, we identified:

- **Critical Transaction Combinations:** Groups of t_codes that, when occurring together, indicate specific system states or potential issues
- **Repetition Patterns:** Normal t_codes that become anomalous when repeated beyond certain thresholds
- **Temporal Patterns:** Time-based patterns where normal transactions become anomalous based on timing
- **Sequential Dependencies:** Transaction sequences that must be monitored as complete workflows

Step 2: Defining Pattern-Based Monitoring Strategies

The monitoring strategies were developed to address:

- **Individual Transaction Monitoring:** Traditional single t_code monitoring with completion times, error rates, and system response times
- **Pattern Recognition:** Monitoring for specific combinations and sequences of t_codes
- **Frequency Analysis:** Detecting when normal transactions occur with unusual frequency
- **Correlation Analysis:** Identifying relationships between seemingly unrelated transactions

Step 3: Establishing Pattern-Based Alert Conditions

Alert conditions were established for:

- **Combination Alerts:** Triggers when specific t_code combinations occur within defined time windows
- **Repetition Alerts:** Notifications when normal t_codes exceed repetition thresholds
- **Sequence Alerts:** Alerts for disrupted or incomplete transaction sequences
- **Temporal Alerts:** Notifications for transactions occurring outside expected time patterns

5.5.2 Chunking Strategy

In addition to embedding generation and storage considerations, implementing a chunking strategy is vital when dealing with large datasets or lengthy text inputs:

What is Chunking?

Chunking refers to breaking down large pieces of text or datasets into smaller manageable segments or “chunks.” This technique is particularly useful when working with machine learning models that have input size limitations or when processing large volumes of transactional data where individual entries must be analyzed separately but collectively contribute to overall insights.

Benefits of Chunking

- **Improved Processing Efficiency:** By breaking down larger datasets into smaller chunks, we can process each segment independently without overwhelming system resources or exceeding memory limits.
- **Enhanced Model Performance:** Smaller inputs allow models like LLMs (Large Language Models) to generate more accurate responses since they focus on fewer details at a time rather than attempting to analyze extensive datasets all at once.
- **Parallel Processing Capabilities:** Chunking enables parallel processing techniques where multiple chunks can be processed simultaneously across different threads or processes—significantly speeding up overall computation time.

5.5.3 Enhanced Chunking Strategy Implementation

When implementing chunking within our system:

- Define appropriate chunk sizes based on model constraints—ensuring that each chunk contains enough context while remaining within input limits.
- Maintain relationships between chunks so that any analysis performed retains awareness of how chunks relate back to original transactions or larger datasets.

This section details the two distinct chunking approaches implemented for the SAP anomaly detection system: adaptive chunking for knowledge graph construction and hierarchical chunking for vector store optimization.

5.5.4 Adaptive Chunking Strategy for Knowledge Graph Construction

The knowledge graph implementation employs an intelligent adaptive chunking strategy that dynamically adjusts chunk sizes based on content characteristics and semantic coherence. This approach ensures optimal knowledge extraction while preserving contextual relationships between SAP transaction patterns.

Core Adaptive Chunking Algorithm

The adaptive chunking algorithm operates with the following parameters:

- **Base chunk size:** 516 tokens (optimized for knowledge extraction)
- **Minimum chunk size:** 128 tokens (ensures sufficient context)
- **Maximum chunk size:** 2048 tokens (prevents information overload)
- **Overlap ratio:** 30% (maintains context continuity)

Content-Aware Size Adjustment

The algorithm dynamically adjusts chunk sizes based on:

Text Complexity Analysis

- High complexity content (complexity > 0.7): Reduces chunk size by 20%
- Low complexity content (complexity < 0.3): Increases chunk size by 20%
- Complexity calculated as unique tokens/total tokens ratio

Content Density Evaluation

- High SAP-specific content density (> 0.5): Reduces chunk size by 10%
- Ensures detailed processing of technical SAP documentation

Semantic Boundary Detection

- Prioritizes sentence boundaries for natural text breaks
- Maintains semantic coherence within chunks
- Calculates semantic coherence scores using spaCy processing

5.5.5 Special Content Handling

The adaptive chunking strategy includes specialized handling for:

- **SAP Notes Pattern Recognition:** Identifies and preserves SAP Note references
- **Image Reference Processing:** Maintains associations between text and visual elements

- **Section Boundary Detection:** Recognizes document structure changes
- **Technical Term Preservation:** Ensures SAP-specific terminology remains intact

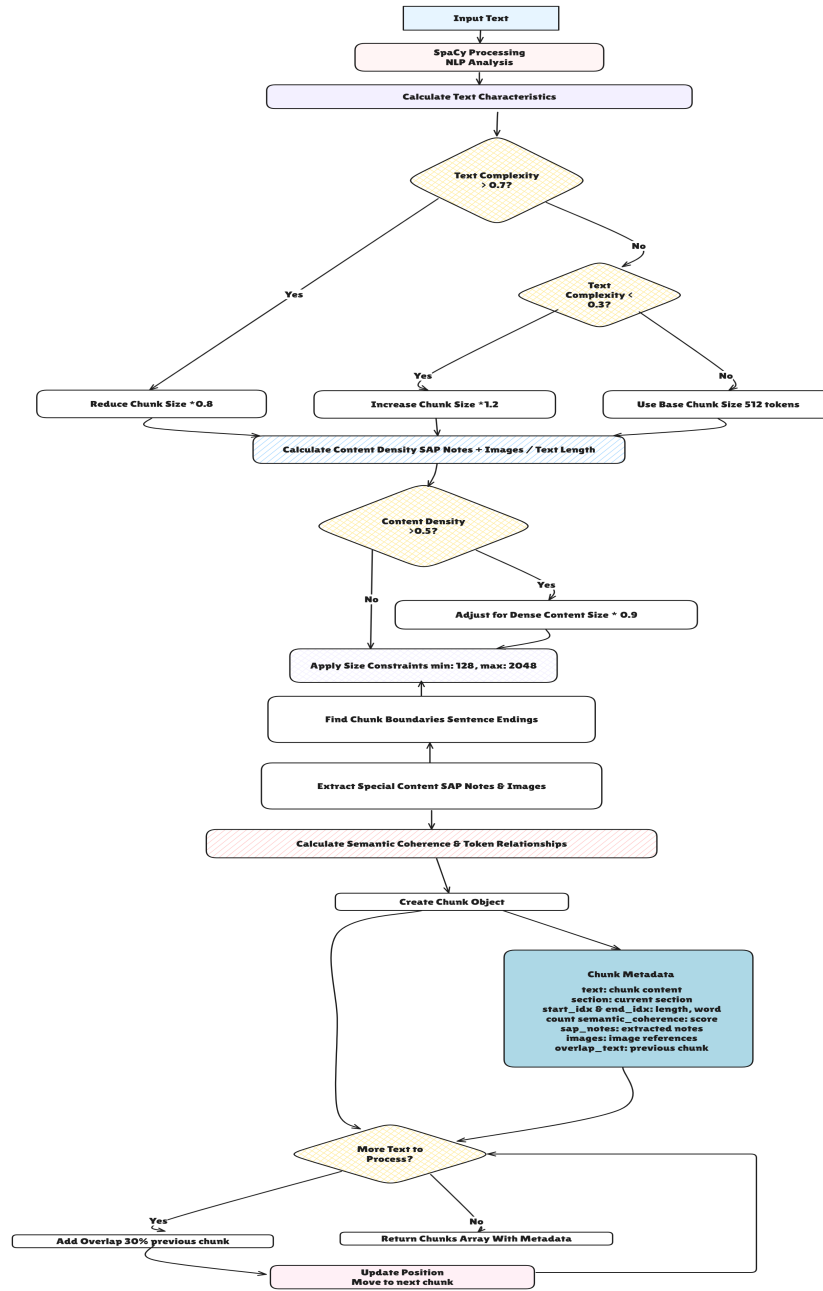


Figure 5.5: Adaptive Chunking Algorithm Flow

5.6 Hierarchical Chunking Strategy for Vector Store

The vector store implementation utilizes a hierarchical chunking approach that preserves document structure while optimizing for retrieval performance. This strategy is specifically designed to handle the complex nested structure of SAP documentation.

5.6.1 Document Structure Preservation

The hierarchical chunking process follows these key principles:

Layout-Aware Extraction

- Maintains heading hierarchies (H1, H2, H3, etc.)
- Preserves parent-child relationships between sections
- Identifies and processes tables as structured HTML
- Recognizes and maintains list structures with proper nesting

Metadata-Aware Processing

- Extracts document metadata (title, creation date, version)
- Preserves structural metadata (section hierarchy, content organization)
- Maintains technical metadata (SAP module associations, transaction code references)

5.6.2 Multi-Level Content Processing

The system processes content at multiple levels:

Block-Level Analysis

- **Heading Detection:** Identifies headings based on position, formatting, and text properties
- **Content Type Classification:** Distinguishes between paragraphs, lists, tables, and images
- **Spatial Analysis:** Uses coordinate information to understand document layout

Content-Type Specific Handling

- **Table Processing:** Converts complex tables to HTML format when enabled
- **List Consolidation:** Combines related list items into coherent chunks
- **Image Association:** Links captions and descriptions to relevant images

5.6.3 Chunk Size Optimization

The vector store chunking strategy employs:

- **Target chunk size:** 768 tokens (optimized for embedding models)
- **Chunk overlap:** 50 tokens (ensures context continuity)
- **Adaptive splitting:** Uses RecursiveCharacterTextSplitter with smart separators
- **Boundary optimization:** Prioritizes natural text boundaries (paragraphs, sentences)

5.6.4 Quality Assurance Measures

The chunking implementation includes several quality assurance mechanisms:

Content Validation

- Ensures minimum content thresholds
- Validates chunk completeness

Duplicate Detection

- Generates deterministic chunk IDs using content and metadata hashing

5.6.5 Comparative Analysis: Adaptive vs. Hierarchical Chunking

Aspect	Adaptive Chunking (Knowledge Graph)	Hierarchical Chunking (Vector Store)
Primary Goal	Knowledge extraction and relationship mapping	Optimal retrieval and semantic search
Chunk Size Strategy	Dynamic based on content complexity	Fixed with smart boundary detection
Content Processing	Linguistic and semantic analysis	Structure-aware document processing
Metadata Focus	Semantic coherence and SAP-specific content	Document hierarchy and positional context
Overlap Handling	Adaptive overlap based on content density	Fixed overlap for context preservation
Boundary Detection	Sentence-level semantic boundaries	Document structure boundaries

While chunking offers numerous benefits, it also presents challenges such as ensuring continuity between chunks during processing or potentially losing context if not managed carefully.

Strategic Organization

To prevent information overload from multiple data sources, we organized each document around specific themes related to transaction patterns rather than individual t_codes. This approach ensures that users can easily find relevant information without getting bogged down in excessive information. We implement it with a straightforward structure:

- **Overview:** A brief summary combining transaction pattern purpose from Help Portal documentation with operational context from real-time data.
- **Pattern Recognition Criteria:** Parameters and thresholds derived from historical data analysis and documentation best practices for identifying anomalous patterns.

- **Alert Conditions:** Specific criteria combining statistical analysis of real-time data with known issue patterns from SAP Notes.
- **Examples:** Case studies of normal vs. anomalous patterns validated through multiple data sources.

Clear Definitions & Examples

Each document includes explicit definitions of technical terms extracted from SAP Help Portal documentation, ensuring that users are not overwhelmed by jargon. Practical examples are included that demonstrate how the defined criteria apply in real-world scenarios, validated through historical transaction data and documented issue patterns.

Example: "Anomalous repetition pattern" is defined as any sequence of normally acceptable Tcodes that occurs more than 3 times within a 10-minute window, based on statistical analysis of PyRFC-extracted historical data and validated against SAP Help Portal performance guidelines and consultant expertise. For example, SM37 followed by ST22 occurring 4 times in 8 minutes would trigger a pattern-based alert, even though each individual transaction is normal.

Conclusion

The insights gained during this phase, including the establishment of pattern-based monitoring parameters and anomaly definitions are valuable to the subsequent modeling strategies chosen.

MODELING AND EVALUATION

This chapter will present the modeling pipeline we used for the monitoring system for SAP Tcodes. We will explain the design choices and reasoning behind each component, including the usage of both NLP, LLM and the selection of models, retrieval strategies and databases.

6.1 Model Architecture and Pipeline

The extraction of structured knowledge is central to our project, generating intuitive and clear explanations for the users is indispensable to every intelligent system. That is exactly why we shall begin this chapter by evoking the capabilities and architecture of the core model used: LLaMA.

LLaMA: Advanced Decoder-Only Implementation

LLaMA (Large Language Model Meta AI) represents a notable advancement in open weights Language Models. The LLaMa models are designed to be small and efficient and incorporating several architectural optimizations over standard transformer implementations.

Justification for Decoder-Only Architecture in RAG Systems

The choice of decoder-only architecture for our SAP t_code monitoring system was motivated by these key considerations that align well with the requirements of RAG tasks:

Autoregressive Generation Advantages: Decoder-only models are top-notch at sequential text generation, a necessity for producing coherent concise explanations and providing users step-by-step resolutions. This autoregressive nature enables the models to build upon previously generated tokens, ensuring logical flow in complex SAP troubleshooting scenarios:

$$P(\text{response}|\text{context}, \text{query}) = \prod_{i=1}^n P(\text{token}_i|\text{context}, \text{query}, \text{token}_{<i}) \quad (6.1)$$

Simplified Architecture for Inference: Decoder-only architecture is preferred due to it eliminating the cross-attention complexity between encoder and decoder components. This simplification helps in reducing computational overhead during inferences, a feature of high priority for real-time SAP alert processing where response latency impacts system reliability.

Superior Context Integration: The unified attention mechanism in decoder-only models allows for seamless integration of the context that was retrieved in the previous phase with the generation process. Something distinguishable in RAG applications where the model must simultaneously attend to:

- Retrieved knowledge base documents
- Recent SAP alert information
- Historical error patterns
- User query context

Scalability Considerations: Decoder-only models have proved to do better at scaling properties with increased parameter counts, this is backed by the success of the GPT-family models.

Model Specifications

For this project, we will only consider the models with large context length.

Table 6.1: *LLaMA Model Variants for RAG Tasks - Technical Specifications and Hardware Requirements*

Model	Parameters	Layers	Hidden Size	Attention Heads	Context Length	Hardware Requirements (Inference) (VRAM / System RAM)
Llama 3.2 3B	3B	N/A	N/A	N/A (GQA)	128K	Min: 4-6GB VRAM (quantized) / 16GB RAM Rec: 8GB VRAM / 16-32GB RAM
Llama 3.1 8B	8B	32	4096	32 (8 GQA)	128K	Min: 8GB+ VRAM (quantized) / 16-32GB RAM Rec: 16-24GB VRAM / 32GB+ RAM
Llama 3 70B	70.6B	80	8192	64 (8 GQA)	8192	Min: 40GB VRAM (Q4 quantized) / 64GB+ RAM Rec: 2x 24GB VRAM or 1x 80GB VRAM / 128GB+ RAM (e.g., 2x RTX 4090, H100)
Llama 3.3 70B	70B	N/A	N/A	N/A (GQA)	128K	Min: 45-53GB VRAM (quantized) / 64-128GB RAM Rec: 1x H100 80GB or 2x RTX 4090 24GB / 128GB-256GB+ RAM

6.2 Knowledge Graph Construction and Management

Knowledge graphs provide structured representation of SAP system entities, transactions, and their complex Inter and Intra dependencies. knowledge graphs enable flexible schema evolution and natural representation of multi-hop transaction flows, user interactions, and system dependencies across heterogeneous SAP landscapes.

SAP-Specific Graph Theoretical Foundations

For SAP transaction monitoring, a knowledge graph can be formally defined as a directed labeled multigraph $G_{SAP} = (V_{SAP}, E_{SAP}, L_{SAP}, \tau, \omega)$ where:

- V_{SAP} represents entities including: {Users, Transactions, Tables, Programs, Systems, Servers, Authorizations}
- $E_{SAP} \subseteq V_{SAP} \times V_{SAP}$ represents relationships: {executes, accesses, depends_on, triggers, authorizes}
- L_{SAP} assigns semantic labels with SAP-specific ontologies
- $\tau : E_{SAP} \rightarrow L_{SAP}$ maps edges to relationship types
- $\omega : E_{SAP} \rightarrow \mathbb{R}^+$ assigns weights for transaction frequency, criticality, or performance metrics

SAP knowledge graphs follow extended RDF triple formats with temporal and performance annotations:

$$\text{SAP_Triple} = (\text{Subject}, \text{Predicate}, \text{Object}, \text{Timestamp}, \text{Performance_Metrics}) \quad (6.2)$$

6.2.1 Graph Database Technologies

Modern graph databases provide optimized storage and query capabilities for large-scale knowledge graphs.

Comparison of Graph Database Models

Table 6.2: Comparison of graph database technologies

Model	Examples	Query Language	Key Features
Property Graph	Neo4j, JanusGraph	Cypher, Gremlin	Flexible schema, node/edge properties
RDF Triple Store	Apache Jena, Virtuoso	SPARQL	W3C standards, reasoning, ontologies
Hybrid Models	Amazon Neptune, Blazegraph	SPARQL, Gremlin	Multi-model support, scalable

Graph Database Selection and Justification

Based on the comparison of graph database technologies in 6.2, we decided to work with Neo4j for this project due to its robust property graph model, Cypher query language support, and seamless integration with enterprise monitoring applications. Unlike RDF triple stores, Neo4j grants us the possibility to represent SAP-specific transaction semantics with rich metadata, and to implement flexible graph traversals for RCA (Root Cause Analysis). Another Major reason is the existence of Neo4j Aura, a fully managed, cloud-based graph database service, while Neo4j Desktop allows for local, self-managed installations. Aura handles infrastructure and operations. The existence of such a choice is valuable for companies.

6.2.2 Graph Neural Networks for Knowledge Graphs

Graph Neural Networks (GNNs) enable learning on graph-structured data through message passing and aggregation mechanisms.

Graph Convolutional Networks (GCN)

GCNs generalize convolution operations to graph structures:

$$H^{(l+1)} = \sigma(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)}) \quad (6.3)$$

where $\tilde{A} = A + I$ is the adjacency matrix with self-loops, $\tilde{D}$ is the degree matrix, and $H^{(l)}$ represents node features at layer l .

Graph Attention Networks (GAT)

GATs employ attention mechanisms to weight neighbor contributions dynamically:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T [W\mathbf{h}_i || W\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}_i} \exp(\text{LeakyReLU}(\mathbf{a}^T [W\mathbf{h}_i || W\mathbf{h}_k]))} \quad (6.4)$$

6.2.3 Knowledge Graph Embeddings

Knowledge Graph Embeddings (KGEs) project entities and relations into continuous vector spaces to allow for scalable reasoning by limiting hallucinations thanks to the relations providing a sure path to the information needed, semantic search, and link prediction.

Comparison of Embedding Techniques

Table 6.3: *Common Knowledge Graph Embedding Models*

Model	Core Idea	Strengths
TransE Bordes et al., 2013	Represents relations as translations in vector space: $h + r \approx t$	Simple and efficient; good for one-to-one relations
DistMult Yang et al., 2015	Bilinear scoring using dot product between entity and relation embeddings: $f(h, r, t) = h^\top \text{diag}(r)t$	Efficient; handles symmetric relations well
ComplEx Trouillon et al., 2016	Extends DistMult to complex-valued embeddings with Hermitian dot product	Captures asymmetric and anti-symmetric relations
RotatE Sun et al., 2019	Models relations as rotations in complex space	Captures symmetry, inversion, and composition of relations
DeepWalk Perozzi et al., 2014	Uses truncated random walks with Skip-Gram (Word2Vec) on nodes	Unsupervised; preserves network structure and proximity
Node2Vec Grover et al., 2016	Enhances DeepWalk with biased random walks to explore diverse neighborhoods	Balances structural and semantic proximity
TransR Lin et al., 2015	Projects entities into relation-specific spaces before applying translation	Models complex relations with separate entity and relation spaces

ComplEx for Semantic Representation

ComplEx [Trouillon et al., 2016](#) embeds entities and relations into a complex vector space and scores triples using a Hermitian dot product. This enables it to model asymmetric, symmetric, and anti-symmetric relations, making it well-suited for real-world graphs and semantic search applications.

Practical Relevance: These embeddings serve as a foundation for tasks that are important to this project such as anomaly detection, advanced retrieval of relevant information, and semantic reasoning over sparse but structured knowledge.

Neo4j Multi-Database Architecture: We implement three specialized Neo4j databases for optimal organization and performance:

1. **SAP Notes Database:** Technical notes and patches
2. **SAP Help Database:** Official documentation structure
3. **Alert Criteria Database:** Monitoring rules and thresholds

PyTorch Geometric Implementation

**Figure 6.1:** *Pygeometric's Logo*[8]

The knowledge graph embeddings were added using PyTorch Geometric (PyG) and general GNN architectures to generate complex embeddings for SAP entities and

relations.

Our hybrid architecture leverages GCN layers for structural feature extraction followed by GAT layers for attention-based neighbor aggregation. The final layer projects node representations into a 128-dimensional complex embedding space, *constrained by available hardware resources while maintaining sufficient representational capacity* for SAP transaction semantics.

The complex embeddings are structured as $\mathbf{e}_i \in \mathbb{C}^{128} = \mathbf{e}_{i,\text{real}} + i \cdot \mathbf{e}_{i,\text{imag}}$, where both real and imaginary components are 128-dimensional real vectors. These embeddings are then integrated with the CompLex scoring function for link prediction and anomaly detection tasks.

Training Results: Our model demonstrated rapid convergence with early stopping implemented at epoch 7. The training loss curve, shown in Figure 6.2, illustrates dramatic initial convergence from 1.0 to approximately 0.4 within the first 15 batches, followed by stabilization around 0.3. The final loss of 0.25 indicates effective learning while the early stopping criterion prevented overfitting.

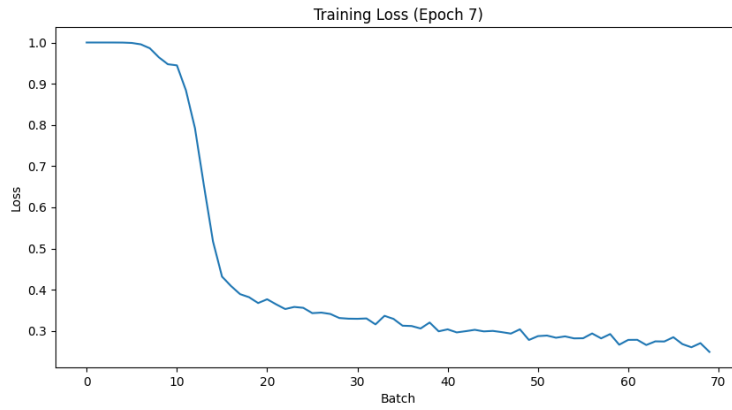


Figure 6.2: Training loss curve showing rapid convergence with early stopping at epoch 7

PyTorch Geometric’s sparse tensor representations and GPU acceleration with CUDA capabilities enabled efficient handling of large-scale SAP knowledge graphs.

6.2.4 Multi-Strategy Triplet Extraction Pipeline

Given the lack of annotated SAP corpora and the complexity of enterprise documentation, we adopt a hybrid approach integrating:

- Transformer-based generation models
- Dependency parsing and syntactic analysis
- Cross-document reasoning with large language models (LLMs)

The Challenge: SAP Documentation Complexity

SAP documentation introduces several challenges that demand the use of multiple recent NLP modules:

- **Technical Jargon:** Hundreds of domain-specific acronyms (e.g., BAPI, tRFC, qRFC, CCMS)
- **Mixed Content:** A combination of tables, code snippets, configuration parameters, and prose
- **Complex References:** Entities referenced across non-contiguous sections
- **Diverse Relationships:** Includes configuration dependencies, monitoring setups, and troubleshooting steps

Table 6.4: *Limitations of Single NLP Models and the Need for a Pipeline*

Challenge	Single Model Limitation	Multi-Model Solution
Entity Recognition	REBEL may miss SAP-specific entities	NER identifies entities; REBEL leverages them
Coreference Resolution	Pronouns like “it” introduce ambiguity	Coreference model resolves references
Domain knowledge	Pre-trained models lack SAP expertise	Domain-specific LLM augments understanding

6.2.5 Component Selection and Evaluation

Coreference Resolution Model Selection

Coreference resolution addresses the linguistic challenge of reference disambiguation. In computational linguistics, this task involves identifying chains of mentions that share the same discourse referent. This step is critical for SAP documentation where technical entities are frequently referenced through pronouns and anaphoric expressions.

Example 1 *Original SAP Documentation:*

“SM37 monitors background jobs. It displays job status and execution details.”

Coreference Chain: “SM37” → “It”

Resolved Text:

“SM37 monitors background jobs. SM37 displays job status and execution details.”

We evaluated the classical and neural approaches:

Table 6.5: Coreference Resolution Models Comparison for SAP Technical Documentation

Model	Approach	SAP Domain Performance	Selection Rationale
Stanford CoreNLP	Rule-based + Statistical	Moderate - struggles with technical terminology	Classical baseline, limited domain adaptation
spaCy NeuralCoref	Neural with pre-trained embeddings	Good - handles general technical terms	Balanced performance but memory intensive
LingMessCoref ¹	Transformer-based with linguistic features	Excellent - best for technical documents	Selected: Fast with Superior handling of technical entity chains
AllenNLP Coref	End-to-end neural	Good performance	Resource intensive for our pipeline

LingMessCoref was selected due to its superior performance on technical documentation, with CUDA acceleration for real-time processing and CPU fallback for deployment flexibility.

Dynamic Acronym Recognition and Resolution

Thanks to the intricate nature of SAP Documentation, acronym processing is critical due to the inconsistent formatting and context-dependent usage. Our dynamic matching system addresses these variations:

Table 6.6: SAP Acronym Format Variations and Handling Strategies

Pattern Type	Examples	Matching Strategy
Standard Definition	Business Application Programming Interface (BAPI)	Regex: <code>([A-Z][a-z\s]+)\((([A-Z]{2,})\)</code>
Standalone Usage	Configure the RFC destination	Dictionary lookup with pluralization
Adjacent Acronyms	LUW and MTE processing	Regex Pattern with conjunctions
Contextual References	The tRFC monitor shows...	Context-aware fuzzy matching

Dictionaries used: ABAP Glossary, SAP Glossary.

Table Processing with Table-to-Text Generation diverse table formats are found across our documents, many of which include hierarchical structures or mixed content

¹ Otmazgin, S., Cattani, A., & Goldberg, Y. (2022). LingMess: Linguistically Informed Multi-Task Instruction Tuning. arXiv preprint arXiv:2204.07178.

² Acronym dictionaries compiled from: SAP ABAP Dictionary, SAP Technical Glossary, and SAP NetWeaver Documentation.

that are not easily interpretable through conventional rule-based or markdown-style extraction. While markdown-based table parsing seemed like a lightweight solution for well-formatted, simple tables, after evaluating the output, it showed signs of struggling with structural inconsistencies and lacks the necessary understanding. To address this, we decided to take a chance with **Table Pre-trained Language Models (TPLMs)** for table-to-text generation. TPLMs are fine-tuned on tasks such as ToTTo, optimized to generate coherent, context-rich descriptions of table content.

Table 6.7: *SAP Table Types and Corresponding Processing Methods*

Table Type	Content Characteristics	Processing Method	Model Used
Configuration Tables	Parameter-value pairs with technical descriptors	Direct extraction with relationship mapping	Rule-based + NER
Transaction Code Tables	T-codes with descriptions and usage contexts	Structured extraction with cross-referencing	Template Matching
Error Code Tables	Error codes, messages, and resolutions	Sequential parsing with SAP Note linking	TPLM+ NER
Process Flow Tables	Step-by-step sequences with conditions	Natural language generation for workflow interpretation	TPLM + Dependency Parsing
Mixed Content Tables	Embedded SAP notes and hybrid structures	Content type classification with hybrid strategy	TPLM + Content Classifier

The BLOOM TPLM model was selected over alternatives due to its superior performance on technical tabular data:

Table 6.8: *Table-to-Text Model Evaluation for SAP Content*

Model	Technical Accuracy	Context Preservation	SAP Domain Relevance	Resource Efficiency
BLOOM-560M-ToTTo	High	Excellent	Good	Excellent
T5-base-table-to-text	Medium	Good	Fair	Good
TAPAS	High	Fair	Poor	Medium

Named Entity Recognition Model Selection NER is the task that identifies and classifies named entities in unstructured text into predefined categories such as persons, organizations, locations, dates, and domain-specific entities.

Since our documentation is heavily technical, we evaluated multiple NER approaches considering domain-specific terminology and multilingual capabilities:

³ Table Pre-trained Language Model: Narrativaai/bloom-560m-finetuned-totto-table-to-text, optimized for table-to-text generation tasks.

⁴ FacebookAI/xlm-roberta-large-finetuned-conll03-english - Selected for balanced performance across technical terminology and processing efficiency.

Table 6.9: Named Entity Recognition Models for SAP Technical Content

Model	Technical Entity Recognition	Multilingual Support	SAP Domain Adaptation	Processing Speed
XLM-RoBERTa-NER ⁴	Excellent	Native	Good	Fast
spaCy Transformer (trf)	Excellent	Excellent	Fair	Medium
BERT-NER	Good	Limited	Fair	Medium
WikiNeuralMultilingual	Good	Excellent	Poor	Slow
DistilBERT-NER	Fair	Limited	Fair	Very Fast

6.2.6 Relation Extraction Models

Comparative Analysis of Triplet Extraction Methods

To support the construction of a domain-specific knowledge graph from SAP documentation, we evaluated three state-of-the-art relation extraction (RE) frameworks: REBEL, WikiNEuRal, and RELIK. Each was assessed based on its architectural principles, domain adaptability, and performance on SAP-specific documents. Our analysis focused on semantic precision, technical term recognition, relationship diversity, and system efficiency.

REBEL (Relation Extraction By End-to-end Language generation)

Architecture & Approach

- **Base Model:** BART-based sequence-to-sequence transformer
- **Method:** Text-to-text generation converting sentences into explicit relational triplets
- **Relation Coverage:** 200+ different relation types
- **Training:** Pre-trained on Wikipedia and Wikidata with fine-tuning capabilities

SAP Documentation Performance Analysis:

Criteria	Analysis
Technical Terminology Recognition	Moderate performance on SAP-specific terms
Relationship Diversity	Excellent coverage of relation types
Processing Speed	Moderate speed due to seq2seq generation
SAP Domain Adaptation	Requires fine-tuning for SAP content
Memory Efficiency	Resource-intensive

Strengths:

- Schema-free and end-to-end
- Capable of extracting technical relationships (dependencies, configurations)
- Handles documentation patterns effectively

Limitations:

- Needs SAP domain fine-tuning
- Sensitive to token length limits
- May hallucinate relations
- Requires post-processing for T-code alignment

WikiNEuRal**Architecture & Approach**

- **Base Model:** Multilingual transformer-based NER
- **Method:** Named Entity Recognition with relation inference
- **Coverage:** General-purpose entities, multilingual
- **Training:** Wikipedia-based

SAP Documentation Performance Analysis:

Criteria	Analysis
Technical Terminology Recognition	Poor SAP term detection
Multilingual Support	Excellent support
Processing Speed	High-speed recognition
SAP Domain Adaptation	Not domain-optimized
Relation Extraction	Limited relationship inference

Assessment: Not recommended for primary SAP triplet extraction. Better as a supplemental NER module.

RELIK (Retrieve and Link)**Architecture & Approach**

- **Components:** Retriever + Reader
- **Method:** Entity linking with RE
- **Target KB:** Wikipedia-based
- **Innovation:** Lightweight, fast, academic-budget optimized

SAP Documentation Performance Analysis:

Criteria	Analysis
Technical Terminology Recognition	Good linking performance
Relationship Extraction	Strong RE via EL
Processing Speed	Optimized
SAP Domain Adaptation	Promising with tuning
Resource Efficiency	Highly efficient

Strengths:

- Fast, scalable processing
- Combined EL and RE
- Suitable for production environments

Limitations:

- Requires SAP-specific KB
- Limited Wikipedia-based coverage

6.3 Cross-Chunk Inference Strategy Analysis

Current Strengths

Implementation Sample:

```
def infer_cross_chunk_relations(triplets, context, all_sections_text):  
    # Zero-shot prompting with domain-specific instructions  
    # JSON validation with robust error handling
```

Strengths:

- Domain-specific prompts
- Full-document context
- Quality filters
- Robust JSON parsing

6.3.1 Composite Pipeline Design: Multi-Strategy Triplet Extraction

To overcome limitations of limiting the usage of a single model, an ensemble approach is integrated to resolve that:

Pipeline Components:

- **REBEL Model** (IBM-research/knownl-large): Primary extraction using text-to-text generation
- **Domain-Specific LLM**: Ollama Llama3.2 for SAP-specific relationship inference
- **Dependency Parsing**: spaCy-based syntactic analysis for supplementary relationships
- **Cross-Document Inference**: LLM-powered relationship discovery across sections

This approach is ideal and robust for the extraction of relational knowledge.

Prompting Strategy and Optimization

Prompt Engineering Methodology We systematically evaluated different prompting approaches to optimize relationship extraction quality.

Few-Shot vs. Zero-Shot Analysis

- **Few-Shot Approach:**

- *Advantage:* Higher initial accuracy for domain-specific patterns
- *Disadvantage:* Model bias toward example patterns, reduced diversity

- **Zero-Shot Approach:**

- *Advantage:* Greater relationship diversity and adaptability
- *Disadvantage:* Requires more sophisticated prompt engineering

Final Prompt Design We adopted a zero-shot approach with structured instructions targeting five key relationship categories:

Your task is to extract HIGH-QUALITY, CONCISE, and ACTIONABLE technical relationships from SAP text for expert-level consultants.

Instructions for RELATION EXTRACTION:

1. Identify DIRECT, TECHNICAL relationships between SAP entities
2. Extract PRECISE, VERB-BASED relations describing interactions
3. Prioritize relations that describe:
 - * ACTIONS performed on/by SAP entities
 - * FUNCTIONS or CAPABILITIES
 - * CONFIGURATIONS or PARAMETERS
 - * DEPENDENCIES or INTERACTIONS
 - * ATTRIBUTES or CHARACTERISTICS
4. EXCLUDE generic relationships ("related to", "part of")
5. Output format: JSON array of triplets

This approach yielded higher relationship diversity (30+ unique relation types vs. 15 with few-shot) while maintaining semantic quality.

Quality Assessment and Validation

Without ground truth annotations, we employ intrinsic quality measures and domain-specific validation:

Knowledge Graph Enhancement Pipeline

Post-extraction enrichment incorporates domain-specific SAP elements:

- **SAP Note Integration:** Automatic detection and hyperlinking of SAP Note references using pattern `(?:SAP Note |see |note |Note )(?:#)?(\d{4,7})`
- **Transaction Code Mapping:** T-code identification and functional area classification

text chunked and linked to its relevant section. The edges in this knowledge graph represent meaningful relationships such as "HAS_SECTION," "CONTAINS_CHUNK," and "APPLIES_TO_COMPONENT." The construction of this particular KG is more straightforward and efficient due to the structure of SAP Notes.

The figure below 6.4 represents parts of the KG structure in our Neo4j local notes database, linking documents to their components and corresponding sections.

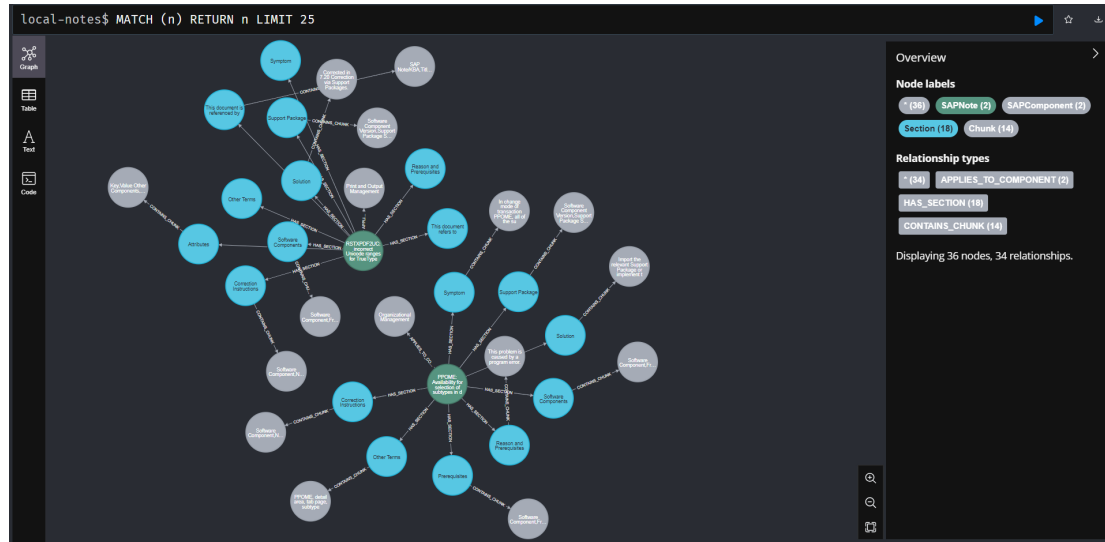


Figure 6.4: Examples from the SAP Notes graph

6.4 Graph Evaluation: LLM-as-a-Judge with G-Eval

To evaluate the quality of the extracted knowledge triplets, we adopted the emerging **LLM-as-a-Judge** paradigm. The idea of this evaluation strategy is to use large language models to assess outputs directly, replacing or augmenting traditional automated metrics. It allows for nuanced, domain-specific judgment aligned with human evaluators.

6.4.1 Motivation for LLM-as-a-Judge

Traditional metrics like BLEU, ROUGE, or METEOR often fail in tasks involving semantic correctness, domain specificity, or factuality. In our scenario—evaluating (subject, relation, object) triplets for SAP transaction codes—subjective dimensions such as *technical accuracy*, *clarity*, and *completeness* are crucial. LLMs, especially those with domain alignment, offer a scalable and consistent alternative to manual evaluation.

6.4.2 Evaluation Framework: G-Eval with Chain-of-Thought Reasoning

Our approach is inspired by **G-Eval**, a structured LLM-based evaluation framework proposed by Liu et al. in “*NLG Evaluation using GPT-4 with Better Human Alignment*”. G-Eval enhances the LLM-as-a-judge approach by decomposing high-level criteria into interpretable reasoning steps.

The G-Eval process involves:

1. **Step Decomposition:** Each evaluation criterion is broken into logical reasoning steps.
2. **Chain-of-Thought Judging:** The LLM generates justifications for each criterion, increasing transparency and alignment.
3. **Scoring and Aggregation:** Numeric scores (1–5 scale) and explanations are produced for each triplet, along with suggestions and global observations.

Tailored Criteria for SAP Triplet Assessment

We adapted the framework to evaluate triplets extracted from SAP documentation. Each triplet captures a factual unit about transaction code behavior or error handling. The evaluation used the Groq-hosted LLaMA 3.3B model with context-truncated documents and focused on the following criteria:

- **Technical Accuracy:** Is the fact correct in the SAP domain context?
- **Relevance:** Is the triplet significant to understanding the transaction?
- **Clarity:** Is the expression easy to interpret and unambiguous?
- **Completeness:** Does the triplet capture the essential relationship fully?

Implementation Details

We evaluated 10 randomly sampled triplets per document. Each evaluation prompt included:

- The document context,
- The list of triplets,
- A JSON template for structured LLM responses.

The model produced:

- Criterion-based scores and reasoning for each triplet,
- A global analysis: issues, suggestions, and summary judgments.

Evaluation Output (Document: Transactional RFC and Queued RFC Monitor):

```
1  {
2    "technical_accuracy": {
3      "score": 4,
4      "justification": "The triplets generally accurately reflect the concepts of
↳ Transactional RFC and Queued RFC, but there are some minor inaccuracies
↳ and potential misunderstandings, such as the implication that
↳ Transaction SM58 guarantees certain attributes of tRFC, when in fact it
↳ is a tool for monitoring tRFC."
5    },
6    "relevance": {
7      "score": 5,
8      "justification": "The triplets are highly relevant to the context of
↳ Transactional RFC and Queued RFC Monitor, as they provide specific
↳ details about the functionality and relationships between different
↳ components of the system."
9    },
10   "clarity": {
11     "score": 4,
12     "justification": "The triplets are generally clear and easy to understand,
↳ but some of the relationships and concepts may require additional
↳ context or explanation for a full understanding, particularly for those
↳ without prior knowledge of SAP systems."
13   },
14   "completeness": {
15     "score": 4,
16     "justification": "The triplets provide a good amount of information about
↳ Transactional RFC and Queued RFC, but there may be some additional
↳ details or relationships that are not included, such as the specific
↳ advantages and disadvantages of using tRFC versus qRFC."
17   },
18   "issues": [
19     "Potential inaccuracies in the implication of Transaction SM58 guarantees",
20     "Lack of additional context for some relationships and concepts"
21   ],
22   "suggestions": [
23     "Provide additional context or explanation for some of the relationships
↳ and concepts",
24     "Clarify the role of Transaction SM58 in relation to tRFC"
25   ],
26   "overall_assessment": "The triplets provide a good foundation for
↳ understanding the concepts of Transactional RFC and Queued RFC, but may
↳ require some additional clarification and context to ensure accuracy and
↳ completeness."
27 }
```

Listing 6.1: *JSON example*

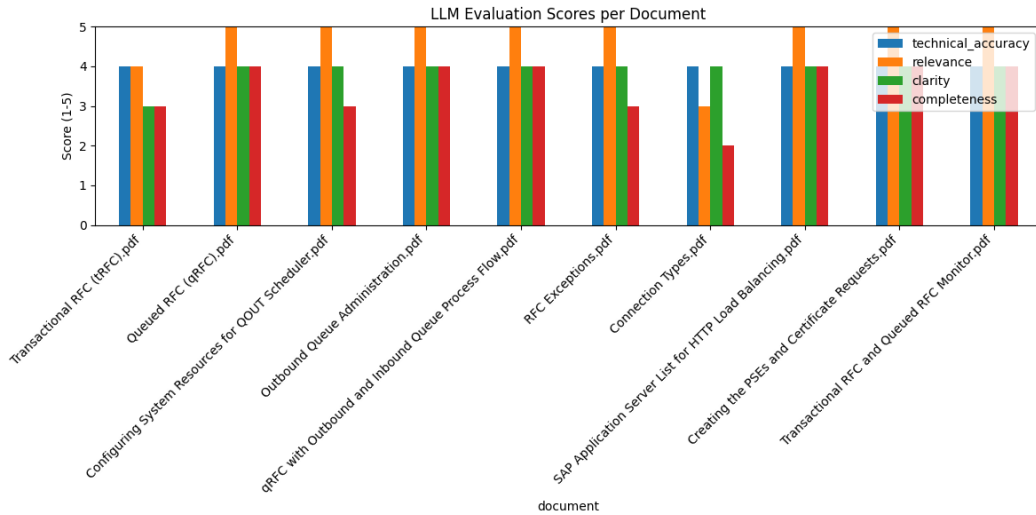


Figure 6.5: Triplet Evaluation Scores using G-Eval on SAP Documents

6.4.3 Advantages Over Other Methods

Compared to metrics like BERTScore, GPTScore, or UniEval, the G-Eval method combined with LLM-as-a-judge principles offers:

- **Better alignment with human judgment** (e.g., Spearman correlation of 0.51 in summarization benchmarks),
- **Interpretability through justification**,
- **Robustness against hallucinations** in factual tasks,
- **Customizability to enterprise-specific documentation and objectives**.

This methodology proved very useful for verifying the factual accuracy and relevance of the extracted SAP knowledge base, particularly where domain nuance matters most.

6.5 Hybrid RAG Approach

Comparison of Approaches

Table 6.11: Comparison of Modeling Approaches for SAP Monitoring

Approach	Advantages	Disadvantages	SAP Suitability
Pure LLM	Fast inference, natural responses	Limited domain knowledge, hallucination risk	Low - lacks relevant SAP knowledge
Traditional Vector RAG	Factual grounding, semantic similarity search	Limited relationship understanding, context fragmentation Wang et al., 2024	Medium - good semantic matching but incomplete context
Graph RAG	Rich structural relationships, causal reasoning	Poor semantic similarity, scalability issues	Medium - strong relationships but weak semantic search
G-Retriever PCST	Efficient subgraph extraction, Prize-Collecting Steiner Tree optimization He et al., 2024	Limited to graph-structured data, complex implementation	High - excellent for structured SAP dependencies
HybridRAG: Vector-First Graph-Enriched RAG Wang et al., 2024	Semantic similarity + structural relationships + SAP-specific optimizations	Most complex implementation, higher computational cost	Highest - comprehensive solution with SAP temporal alignment

6.5.1 Vector-First Graph-Enriched RAG Architecture

This project proposes a specialized RAG pipeline tailored to intelligent SAP system monitoring that combines vector-based semantic retrieval with graph-enhanced reasoning. The approach supports accurate, interpretable, and context-aware alerting. It follows the HybridRAG model Wang et al., 2024, integrating both dense and structured retrieval techniques to enable root cause analysis, optimization, and diagnostics.

The **vector-first** strategy emphasizes semantic similarity while incorporating structured knowledge for enhanced accuracy.

Core Architecture Components

- **Dense Vector Retrieval via Qdrant:** Semantic similarity across logs, SAP Help content, and guides using 768-dimensional Nomic embeddings.
- **Graph Enhancement via Neo4j:** Relationship-aware context expansion using graph traversal.

- **Hybrid Fusion:** Reciprocal rank fusion and weighted combination of results.
- **Multi-Backend Inference:** Inference through local (Ollama) or cloud (GROQ) engines.

6.5.2 Multi-Stage Retrieval Pipeline

This architecture employs a four-stage retrieval pipeline:

Stage 1: Dense Vector Retrieval

1. **Query Understanding:** Detects `t_codes` (e.g., SM37), system IDs (SIDs), alert types, messages, and time references.
2. **Embedding Generation:** Queries encoded with the same encoder used for document embeddings.
3. **Approximate Nearest Neighbor Search:** High-cosine similarity retrieval from vector space.
4. **Context Tagging:** Metadata (severity, SID, timestamp) annotation for each document.

Stage 2: Sparse Retrieval Integration

- TF-IDF/BM25-based exact term matching.
- Fusion of dense and sparse results using reciprocal rank or weighted combination.

Stage 3: Graph-Based Enhancement

Cypher queries to Neo4j enable structural context expansion:

1. **Entity Mapping:** Linking queries to graph nodes.
2. **Graph Traversals:**

```
MATCH (a:Alert)-[:TRIGGERED]->(e:Error)
WHERE e.code = 'SM37'
RETURN a, e
```

3. **Temporal Alignment:** Normalizing event timelines.
4. **Graph Expansion:** Extracting relationships for adjacent or related entities.

Stage 4: Reranking and Selection

- Cross-encoder-based reranking.
- Fusion layer integrates semantic and structural relevance.
- Diversity-aware selection for final result set.
- Optimization for context length with Qdrant-priority ordering.

6.5.3 Knowledge Graph Integration

Graph Database Schema

- **Entities:** Systems, Alerts, Jobs, Errors, Transaction Codes.
- **Relation examples:** TRIGGERED, FAILED_DUE_TO, DEPENDS_ON.
- **Attributes:** Severity, timestamp, SID.
- **Constraints:** Graph consistency and data integrity.

Enhanced Functionality

- **Disambiguation:** Clarifies vague semantic results.
- **Causal Reasoning:** Identifies propagation paths and dependencies.
- **Fallback Logic:** Complements Qdrant and vice versa.

6.5.4 Advanced SAP-Specific Capabilities

Temporal Alignment Impact

- UTC normalization of timestamps.
- Sliding window of ± 5 minutes for correlation.
- Batch job synchronization across landscape tiers.

Cross-System Reasoning

- **Cross-SID Traversal:**

```
MATCH p=(s1:System)-[*1..3]->(s2:System)
WHERE s1.sid='PRD' AND EXISTS((s2)-[:RAISED]->(:Alert))
RETURN p
```

- Causal clustering of events.

6.5.5 Prompt Engineering Strategy

- **Role-Aware Prompts:** Custom for executives, engineers, and Copilot.
- **Context Injection:** Prioritizes Qdrant vector results, followed by Neo4j expansion.
- **Few-Shot Examples:** Improves alert handling performance.
- **Cypher-Informed Reasoning:**

```
Given the results of
MATCH (j:Job)-[:FAILED_DUE_TO]->(e:Error)
WHERE j.tcode='SM37', analyze potential root causes.
```

- **Severity-Aware Prompting:** Adapts tone and granularity based on alert importance.

6.5.6 Evaluation Strategy

Due to the lack of SAP-annotated datasets, we employ a Hybrid RAG-inspired evaluation framework:

- **LLM-as-a-Judge:** Scoring outputs on faithfulness, recall, and relevance.
- **Expert Feedback:** SAP consultants assess and refine prompts.
- **Synthetic Prompts:** Simulate real scenarios (e.g., job delays, import errors).

Evaluation Metrics **Wang et al., 2024:**

- **Faithfulness:** 0.96
- **Answer Relevance:** 0.96
- **Context Recall:** 1.0
- **Context Precision:** ≈ 0.81

Component Integration Flow

1. Query processing and embedding generation
2. Primary retrieval via Qdrant
3. Graph enhancement through Neo4j
4. Hybrid result fusion and ranking
5. LLM-based response generation

To summarize, our hybrid RAG pipeline extends the HybridRAG framework to meet the challenges of SAP monitoring. Through a “vector-first, graph-enhanced” strategy, limiting hallucinations and providing structural grounding. With temporal alignment, multi-hop reasoning, and adaptive prompting, we are able to have relevant alerting and clear diagnostics, even in the absence of labeled training data.

6.5.7 Retrieval Architecture

We have a two-tier retrieval approach designed to prioritize domain-specific knowledge while maintaining comprehensive coverage through standard documentation.

Primary Retrieval: Tailored Knowledge Base

The retrieval system starts by a **cross-collection** querying, starting with **the incoming Tcodes from SAP**, aggregating them by system and timestamps. It then proceeds to query the relevant KBs created in collaboration with SAP BASIS consultants and administrators based on the available Tcodes in our database. These KBs contain:

- client-specific transaction patterns and their expected behaviors
- Customized alert conditions based on system requirements
- Contextual information about critical business processes
- Information about the architecture and component of the client’s SAP systems

This tier ensures that responses are contextually relevant to the specific organizational environment and their business processes.

Secondary Retrieval: SAP Documentation

Following the primary retrieval, the system queries the broader SAP Help Portal and SAP Notes documentation to retrieve:

- General technical documentation for transaction codes
- Standard troubleshooting procedures
- Generic performance benchmarks and configuration recommendations
- Comprehensive technical specifications

This dual-retrieval approach ensures comprehensive coverage while prioritizing organizational context.

Retrieval Fusion Strategy

The system combines results from both tiers using weighted relevance scoring:

$$\text{final_score}(d) = \beta \cdot \text{primary_score}(d) + (1 - \beta) \cdot \text{secondary_score}(d) \quad (6.5)$$

where $\beta > 0.5$ to prioritize tailored knowledge base results.

6.5.8 Vector Database Selection Justification

The selection of an appropriate vector database for our SAP T-codes monitoring and conversational chatbot system requires careful consideration of enterprise-grade requirements, real-time performance constraints, and the specific demands of hybrid RAG architectures in production SAP environments.

Technical Requirements Analysis

Our SAP monitoring system presents challenges that differentiate it from regular vector search applications:

- **Multi-dimensional Filtering:** SAP transaction analysis requires simultaneous filtering by transaction types, temporal windows, and criticality levels.
- **Real-time Ingestion:** Continuous processing of SAP transaction logs, alert updates, and evolving business rules without service interruption
- **Concurrent Access:** Support for multiple BASIS administrators and functional consultants accessing the system simultaneously
- **Enterprise Reliability:** 99.9% uptime requirements for critical business process monitoring
- **Scalable Architecture:** Ability to handle hundred-thousands of daily transactions across multiple SAP instances and systems.

Comprehensive Vector Database Comparison

Table 6.12: *Comprehensive Vector Database Evaluation Matrix*

Database	Architecture	Metadata Filtering	Real-time Updates	Horizontal Scaling	ACID Compliance	Multi-tenancy	Production Readiness
Qdrant	Server/Cluster	Advanced	Incremental	Native	Partial	Collections	High
FAISS	Library	None	Static Rebuild	Manual	None	Manual	Research
ChromaDB	Embedded/Server	Basic	Yes	Limited	SQLite-based	Limited	Medium
Weaviate	Server/Cluster	GraphQL-based	Yes	Native	Partial	Multi-tenant	High
Pinecone	Cloud SaaS	Advanced	Yes	Automatic	Managed	Projects	High
PGVector	PostgreSQL Ext	SQL-based	Yes	PostgreSQL limits	Full ACID	Schema-based	Medium

Detailed Analysis by Solution

FAISS (Facebook AI Similarity Search) While FAISS offers exceptional in-memory performance and algorithmic sophistication, it presents fundamental limitations for enterprise SAP monitoring:

Critical Limitations:

- **No Metadata Support:** Cannot filter by transaction type, user role, or temporal constraints, essential for SAP context
- **Static Index Structure:** Requires complete index rebuilds for updates, incompatible with real-time SAP transaction ingestion
- **Library-Only Architecture:** No server component, requiring custom service layer development

Use Case Mismatch: FAISS excels in research environments and single-process applications but lacks the database features required for production SAP systems.

ChromaDB ChromaDB offers Python-native simplicity but faces scalability constraints for enterprise deployments:

Limitations for SAP Systems:

- **Single-Node Architecture:** Cannot distribute load across multiple servers for large SAP installations
- **Python Performance Bottleneck:** Interpreted language overhead affects query latency under concurrent load
- **Limited Metadata Filtering:** Basic filtering capabilities insufficient for complex SAP business rules
- **Maturity Concerns:** Relatively new with reported stability issues in production environments

Weaviate Weaviate provides robust GraphQL-based querying but introduces complexity overhead:

Analysis for SAP Context:

- **Schema Complexity:** Requires extensive upfront schema design for SAP transaction relationships

- **Memory Overhead:** Higher resource consumption compared to specialized vector databases
- **GraphQL Learning Curve:** Additional complexity compared to other databases.

Pinecone Pinecone offers managed cloud services but presents vendor lock-in concerns:

Enterprise Considerations:

- **Vendor Dependency:** Cloud-only deployment incompatible with on-premises SAP security requirements
- **Cost Scaling:** Usage-based pricing can become prohibitive for high-volume SAP transaction processing

PGVector (PostgreSQL Extension) PGVector leverages PostgreSQL's mature ecosystem but faces performance limitations:

SAP Integration Challenges:

- **Single-Node Scaling:** PostgreSQL's limitations affect large-scale vector operations
- **Index Rebuild Overhead:** Updates require expensive index reconstructions
- **Memory Constraints:** Limited in-memory optimization for vector operations

Qdrant Selection Justification

Qdrant emerges as the optimal solution for our SAP monitoring architecture based on the following technical advantages:

Production-Grade Architecture

- **Rust Performance:** Native performance with memory safety guarantees
- **Horizontal Scaling:** Built-in clustering with automatic shard distribution
- **Incremental Updates:** Real-time vector insertion without index rebuilds
- **Snapshot Consistency:** Point-in-time recovery for enterprise data protection

SAP-Specific Optimizations Our Qdrant configuration leverages specialized features for SAP monitoring:

**FAISS performance without server infrastructure and metadata filtering*

Architectural Integration Benefits

Qdrant's selection enables our hybrid RAG architecture through:

1. **Multi-Collection Strategy:** Separate collections for documentation, internal KB, transaction patterns, and alert criteria
2. **Temporal Awareness:** Time-based partitioning for SAP transaction lifecycle analysis
3. **Hybrid Search:** Vector similarity combined with business rule filtering
4. **Graph Enhancement:** Seamless integration with Neo4j for relationship traversal

Table 6.13: *Qdrant Configuration Optimized for SAP Workloads*

Parameter	Value	SAP-Specific Rationale
HNSW M	32	Balanced connectivity for diverse transaction patterns
HNSW EF Construct	128	High precision for critical alert matching
Quantization	Scalar	Memory efficiency for large transaction volumes
Payload Indexing	JSON	Fast filtering on SAP metadata fields
Distance Metric	Cosine	Optimal for normalized business process embeddings
Replication Factor	2	High availability for critical monitoring

Risk Mitigation Strategy

Qdrant's open-source nature and active community provide:

- **Vendor Independence:** No lock-in with on-premises deployment options
- **Customization Capability:** Source code access for SAP-specific optimizations
- **Community Support:** Active development and enterprise adoption
- **Migration Path:** Standard APIs enable future database transitions if needed

The selection of Qdrant for our application is influenced by its unique combination of production-grade reliability, advanced filtering capabilities, and architectural flexibility. While alternatives like FAISS offer excellent performance, only Qdrant delivers the comprehensive feature set required for the circumstances of the project.

6.5.9 Embedding Model (EmbeddingModel)

We are using Qdrant's FastEmbed integration with Nomic Embed model for high-quality vector representations:

Model Specifications:

- **Model:** nomic-embed-text-v1.5
- **Dimensions:** 768
- **Context Length:** 8192 tokens
- **Specialization:** Optimized for semantic search and clustering tasks

Embedding Generation Process:

$$E_{\text{query}} = \text{NomicEmbed}(\text{preprocess}(Q)) \quad (6.6)$$

Where Q represents the input query and preprocessing includes normalization and tokenization specific to SAP domain terminology.

Temporal Awareness for Transactions: For transaction nodes, we implement temporally-aware embeddings that emphasize date-related fields:

$$E_{\text{temporal}} = \alpha \cdot E_{\text{base}} + \beta \cdot E_{\text{temporal_fields}} \quad (6.7)$$

Where $\alpha + \beta = 1$ and $\beta > 0.3$ to prioritize temporal information in transaction analysis.

6.5.10 Vector Database (VectorDatabase)

Qdrant Implementation: Qdrant serves as our high-performance vector database with the following configuration:

Table 6.14: Qdrant Configuration for SAP Monitoring System

Parameter	Value	Description
Vector Size	768	Matching Nomic Embed dimensions
Distance Metric	Cosine	Optimal for normalized embeddings
HNSW M	32	Balanced connectivity and performance
HNSW EF Construct	128	High-quality index construction
Quantization	Scalar	Memory optimization without accuracy loss
Payload Index	JSON	Efficient metadata filtering

Collection Structure:

- `sap_documentation`: SAP Help Portal content embeddings
- `internal_kb`: Organization-specific knowledge base
- `transaction_patterns`: Historical transaction analysis results
- `alert_criteria`: Alert condition definitions and thresholds

Advanced Retrieval Features:

- **Payload Filtering:** Metadata-based pre-filtering for relevance
- **Hybrid Search:** Combination of vector similarity and keyword matching
- **Multi-Vector Search:** Support for different embedding spaces

6.5.11 Beam Search

Beam Search Optimization: is a graph traversal technique that maintains a fixed-size set of candidate paths during search operations. The algorithm balances exploration and exploitation by controlling the search width through a beam parameter:

Maintains top-k most promising candidates at each exploration step Controls computational complexity vs. search quality trade-off Beam parameter `ef` (exploration factor) determines search breadth. Larger beam sizes improve recall at the cost of increased latency

¹ https://d2l.ai/chapter_recurrent-modern/beam-search.html/

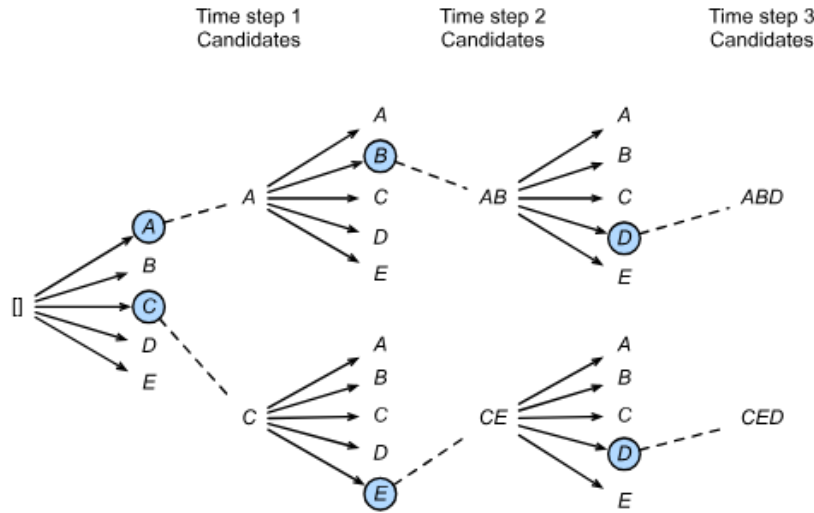


Figure 6.6: The process of beam search (beam size=2) ¹

Contextual Response Generation with Llama 3.2

- We use **Llama 3.2**, a large language model, for response generation.
- **Beam Search Decoding:** To enhance response accuracy, we implemented a beam search with beam width $B = 5$, balancing between response quality and computational efficiency.
- **Temperature Parameter:** A lower temperature ranging from 0 to 0.3 was selected to reduce variability in the output, making the responses more deterministic and consistent for critical alert scenarios.

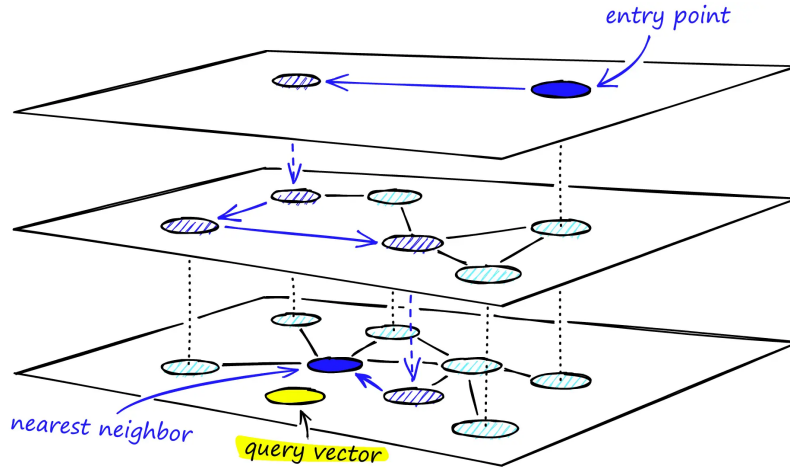
Hierarchical Navigable Small World (HNSW)

HNSW slots into the graph category of the ANN algorithms. More specifically, it is a proximity graph, in which two vertices are linked based on their proximity (closer vertices are linked) — often defined in Euclidean distance. HNSW constructs a multi-layer graph structure enabling logarithmic search complexity:

- Layer construction based on exponential decay probability
- Greedy search with beam optimization
- Trade-off between index construction time and query performance

Adding hierarchy to NSW produces a graph where links are separated across different layers. At the top layer, we have the longest links, and at the bottom layer, we have the shortest.

² <https://www.pinecone.io/learn/series/faiss/hnsw>

Figure 6.7: Layered graph of HNSW²

6.5.12 G-Eval Assessment Framework

To validate the effectiveness of our vector-first graph-enriched RAG system, we implemented a comprehensive evaluation framework using G-Eval methodology. This approach addresses the challenge of evaluating RAG systems in specialized domains like SAP monitoring, where traditional metrics may not capture domain-specific requirements and there is limited availability of annotated datasets.

Evaluation Methodology

Our G-Eval implementation employs a multi-dimensional assessment framework that evaluates four critical aspects of alert system performance:

- **Technical Accuracy:** Measures the presence and correctness of essential SAP-specific identifiers including system IDs, transaction codes (T-codes), severity levels, and timestamp validation
- **Relevance:** Assesses the contextual appropriateness of retrieved alerts, prioritizing high-priority incidents and recent events while penalizing duplicate content
- **Clarity:** Evaluates the readability and structure of alert information, considering optimal content length, organizational structure, and technical jargon density
- **Completeness:** Measures the presence of required metadata fields and optional descriptive information necessary for effective incident response

The overall evaluation score is computed using weighted combination:

$$\text{G-Eval Score} = 0.3 \cdot \text{Technical Accuracy} + 0.25 \cdot \text{Relevance} + 0.25 \cdot \text{Clarity} + 0.2 \cdot \text{Completeness} \quad (6.8)$$

Performance Analysis

Figure 6.8 presents the comprehensive performance analysis of our alert retrieval system across 50 evaluation instances. The results demonstrate several key findings:



Figure 6.8: G-Eval Performance Analysis showing metric distributions, correlations, and overall system performance trends

Metric Performance:

- **Technical Accuracy** achieved the highest average score (4.65 ± 0.36), indicating excellent preservation of SAP-specific technical identifiers and metadata integrity
- **Completeness** scored well (4.06 ± 0.24), demonstrating comprehensive information retrieval across required and optional fields
- **Clarity** showed moderate performance (3.78 ± 0.78), with higher variance suggesting inconsistent content structure across different alert types
- **Relevance** exhibited the lowest scores (2.65 ± 0.87) with high variance, indicating challenges in contextual matching and priority assessment

System Stability: The overall performance trend analysis reveals consistent system behavior with scores ranging between 3.0-4.5, demonstrating stable retrieval quality across diverse query contexts. The slight downward trend (shown by the dashed regression line) suggests the need for continuous model refinement.

Metric Correlations: The correlation heatmap reveals minimal interdependence between evaluation metrics (correlation coefficients ≤ 0.24), indicating that our multi-dimensional assessment captures distinct aspects of system performance. This validates the comprehensiveness of our evaluation framework.

Implications for SAP Monitoring

The G-Eval results validate the effectiveness of our hybrid RAG approach for SAP alert management:

- **Strong Technical Foundation:** High technical accuracy scores confirm successful integration of domain-specific knowledge graphs with vector retrieval
- **Completeness Achievement:** Excellent completeness scores demonstrate the system's ability to provide comprehensive context for incident resolution
- **Areas for Enhancement:** Lower relevance scores highlight opportunities for improved query understanding and contextual ranking algorithms
- **Consistent Performance:** Stable performance across evaluations indicates robust system architecture suitable for production deployment

This evaluation framework provides a foundation for continuous system improvement and validates our approach's suitability for intelligent SAP monitoring applications.

6.6 Chatbot Implementation

The **SAP Basis Assistant** chatbot is an advanced conversational AI system designed to support SAP system monitoring and troubleshooting. NLP, knowledge integration, and system context-awareness to deliver intelligent responses to end-users.

Natural Language Understanding and Query Modeling

The chatbot extracts key entities from user queries such as the SAP System ID and transaction codes (T-codes), a crucial step for tailoring the response to the appropriate system context.

Conversational Memory

We store and retrieve past messages using a session-based memory system.

Query Reformulation

To enhance the quality of the semantic search and LLM output processes, the user queries are reformulated based on chat history and the identified system's context.

Listing 6.1: *Query Reformulation with Context*

```
def _reformulate_query_with_history(question, session_id):  
    ...
```

Capabilities and Reasoning Mechanisms

The chatbot is capable of:

- Real-time SAP system monitoring (e.g., ST22, SM37)
- Transaction code explanation and interpretation
- Knowledge synthesis across multi-source inputs

It uses keyword grounding and semantic filtering to reason over similar past cases, historical alerts, and defined resolutions.

The chatbot architecture is made up of modular components that handle query understanding, system context detection, knowledge retrieval, and natural language generation. Figure 6.9 explains the main components and how they interact.

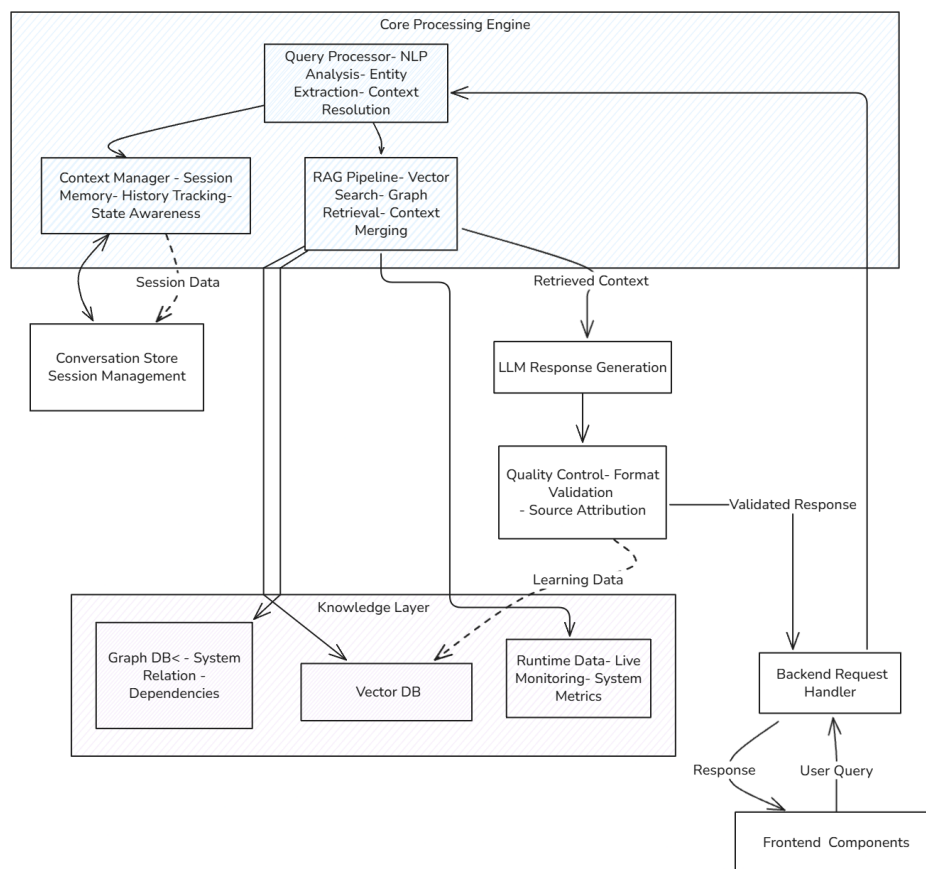


Figure 6.9: High-level architecture of the SAP Basis Assistant chatbot

Conversation Model (ChatModel)

Multi-Backend Architecture: Our system supports flexible model selection:

Table 6.15: Used Language Models and Specifications

Backend	Model	Context Length	Latency	Use Case
Ollama	Llama 3.2 3B	131K	Medium	Development, Privacy
GROQ	deepseek-r1-distill-llama-70b	128K	Very Low	Production, Complex queries
GROQ	llama-3.3-70b-versatile	128k	Low	Balanced performance

Evaluation Metrics The system evaluates responses based on:

- Relevance to query
- Faithfulness to source data
- Coherence of response
- Completeness of information

Test Scenarios:

- SAP transaction error diagnosis
- Alert interpretation and prioritization
- Resolution step generation
- Technical documentation synthesis

Conclusion

Combining Qdrant vector database, Neo4j knowledge graphs, and hybrid inference backends,By means of intelligent retrieval fusion, temporal-aware embeddings, and flexible model selection, the solution shows a promising performance, thereby laying a strong base for AI-based SAP system monitoring and alert management.

DEPLOYMENT

This chapter outlines the complete deployment pipeline of the solution, integrating all components of the system from frontend interaction to backend processing, LLM inference, and feedback/alerting mechanisms.

7.1 Integration with SAP Data Sources

The foundation of this project are the Tcodes that are coming from SAP systems and accessed from the company's Horizon VMWare machines. This data is captured and processed through custom APIs and scripts that were developed by the team's SAP Administrators, collecting real-time logs and transaction data.

Our role here was to change the formats of the data to be then stored in Qdrant and Neo4j collections that capture both current transaction states and historical logs.



Figure 7.1: VMware Horizon Client logo[10]

Overview

The deployment architecture is designed for scalability, modularity, and user customization. The system allows switching between local and cloud inference engines (Ollama or GROQ), and supports integration with both frontend interfaces and enterprise messaging tools like Microsoft Teams. Figure 7.2 presents the overall deployment diagram. Visual Studio Code [9] was used as the primary code editor.

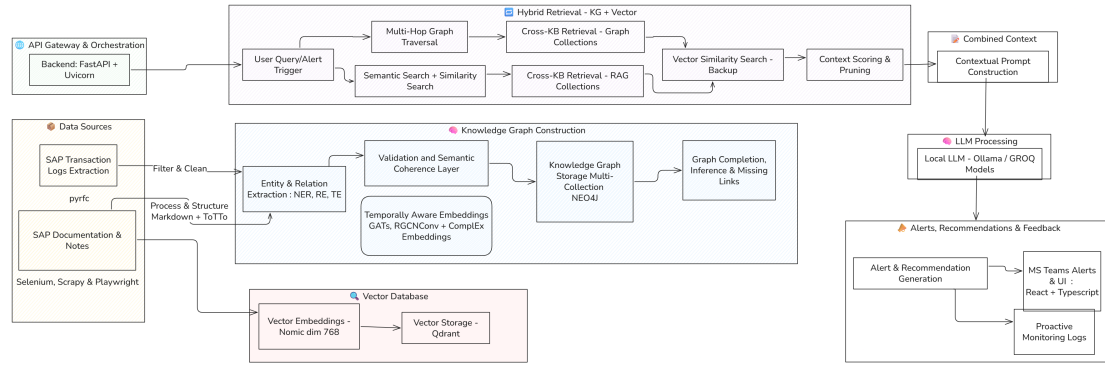


Figure 7.2: End-to-end Deployment Architecture

7.2 Backend Infrastructure

The backend is implemented using FastAPI and served with Uvicorn.



Figure 7.3: Technology stack for serving RESTful APIs

It exposes RESTful APIs for various services including:

- Embedding generation using HuggingFace Transformers
- RAG retrieval pipeline
- Chatbot conversation handler
- Alert and evaluation modules

The system uses asynchronous endpoints for efficient I/O handling and includes middleware for logging, monitoring, and authentication.

7.3 Frontend Interface

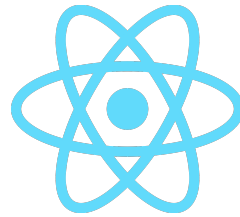
The frontend is built with React and TypeScript, offering a rich UI for:

- Submitting queries
- Viewing retrieved chunks and knowledge graph context
- Interacting with the chatbot
- Receiving system alerts and validation messages

The UI communicates with the backend using REST APIs and WebSockets for real-time features.



TypeScript
[13]



*React: UI library for building inter-
faces*[14]

Figure 7.4: Frontend technologies used in the web interface

7.4 Chatbot and Evaluation Integration

The chatbot is integrated into the UI, connected to the same backend LLM inference and RAG pipeline. G-Eval is embedded into the chatbot loop to:

- Evaluate the quality of triplets
- Score the factual correctness of responses
- Provide feedback to the user

This enables dynamic quality checking during interaction.

7.5 Authentication and Security

7.5.1 JWT-Based Authentication

The system implements JSON Web Token (JWT) authentication with refresh token functionality for secure access control:

Listing 7.1: *JWT Authentication Implementation*

```
class AuthService:
    def __init__(self):
        self.SECRET_KEY = os.getenv("SECRET_KEY")
        self.ALGORITHM = "HS256"
        self.ACCESS_TOKEN_EXPIRE_MINUTES = 30
        self.REFRESH_TOKEN_EXPIRE_DAYS = 7
```

Key security features include:

- Token-based session management with refresh mechanisms
- Bcrypt password hashing with configurable rounds
- Rate limiting for login attempts
- Cache invalidation for logged-out sessions

7.5.2 Session Management

Session management is implemented using a combination of JWT and cache-based validation:

Listing 7.2: *Session Cache Implementation*

```
class CacheManager:
    def __init__(self, default_ttl: int = 300):
        self.cache: Dict[str, Dict[str, Any]] = {}
        self.default_ttl = default_ttl
        self.lock = Lock()
```

7.6 Caching and Performance Optimization

7.6.1 Multi-Level Caching Strategy

The system implements a sophisticated caching mechanism to optimize performance:

- Vector search results caching (TTL: 10 minutes)
- System health data caching (TTL: 1 minute)
- Authentication token caching (TTL: 30 minutes)
- Transaction monitoring results (TTL: 2 minutes)

7.6.2 Cache Invalidation

Intelligent cache invalidation is implemented based on:

- Time-based expiration
- User logout events
- System updates
- Manual refresh requests

7.7 LLM Inference Engines and Integration

Users can select between two inference backends:

7.7.1 Ollama (Local Inference)

Ollama is a framework that is both lightweight and extensible suitable for building and the offline usage of quantized language models on a local machine. It ensures data privacy and fast inference in controlled environments.

7.7.2 GROQ (Cloud Inference)

GROQ cloud provides swift inference using hosted LLMs. It represents a very good solution for latency-sensitive and compute-intensive use cases.

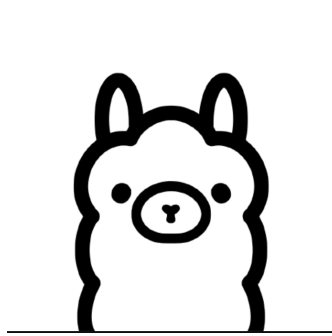


Figure 7.5: Ollama's logo[15]



Figure 7.6: Groq's Logo[16]

7.7.3 Model Selection Justification

The choice between Ollama and GROQ was based on:

Table 7.1: Comparison between Ollama and GROQ inference engines

Local vs Cloud LLM Inference Engines	
Ollama (Local)	GROQ (Cloud)
Deployment Type Local (offline)	Deployment Type Cloud-hosted
Key Benefits • Full data privacy • No internet dependency • Low-latency inference • Cost-effective for scale	Key Benefits • High performance for complex tasks • No local GPU needed • Automatic updates • Scalable infrastructure
Model Used LLaMA 3.2 - 3B parameters Temperature: 0	Model Used LLaMA 3.3 - 70B Versatile Temperature: 0
Model Specifics • Context window: 132k tokens • Max loaded models: 8 • Parallelism: 8 threads • Multilingual support • Free and open-source	Model Specifics • Context window: 128K tokens • Max completion: 32,768 tokens • Gen speed: 275 tokens/sec • Input: \$0.59/1M tokens • Output: \$0.79/1M tokens

7.8 Deployment Configuration

Environment Configuration

Key environment variables for deployment:

Listing 7.3: *Environment Configuration*

```
# LLM Configuration
GROQ_API_KEY=<api_key>
OLLAMA_HOST=http://localhost:11434

# Database Configuration
QDRANT_CLOUD_URL=<url>
QDRANT_API_KEY=<api_key>
NEO4J_URI=bolt://localhost:7687

# MS Teams Configuration
MS_TEAMS_WEBHOOK_URL=<webhook_url>

# Security
SECRET_KEY=<jwt_secret>
```

7.9 User Interface and Monitoring Workflows

Figure 7.7 illustrates the interaction flows between all of our system's components, demonstrating how authentication, monitoring, chat interactions, and alert notifications work together.

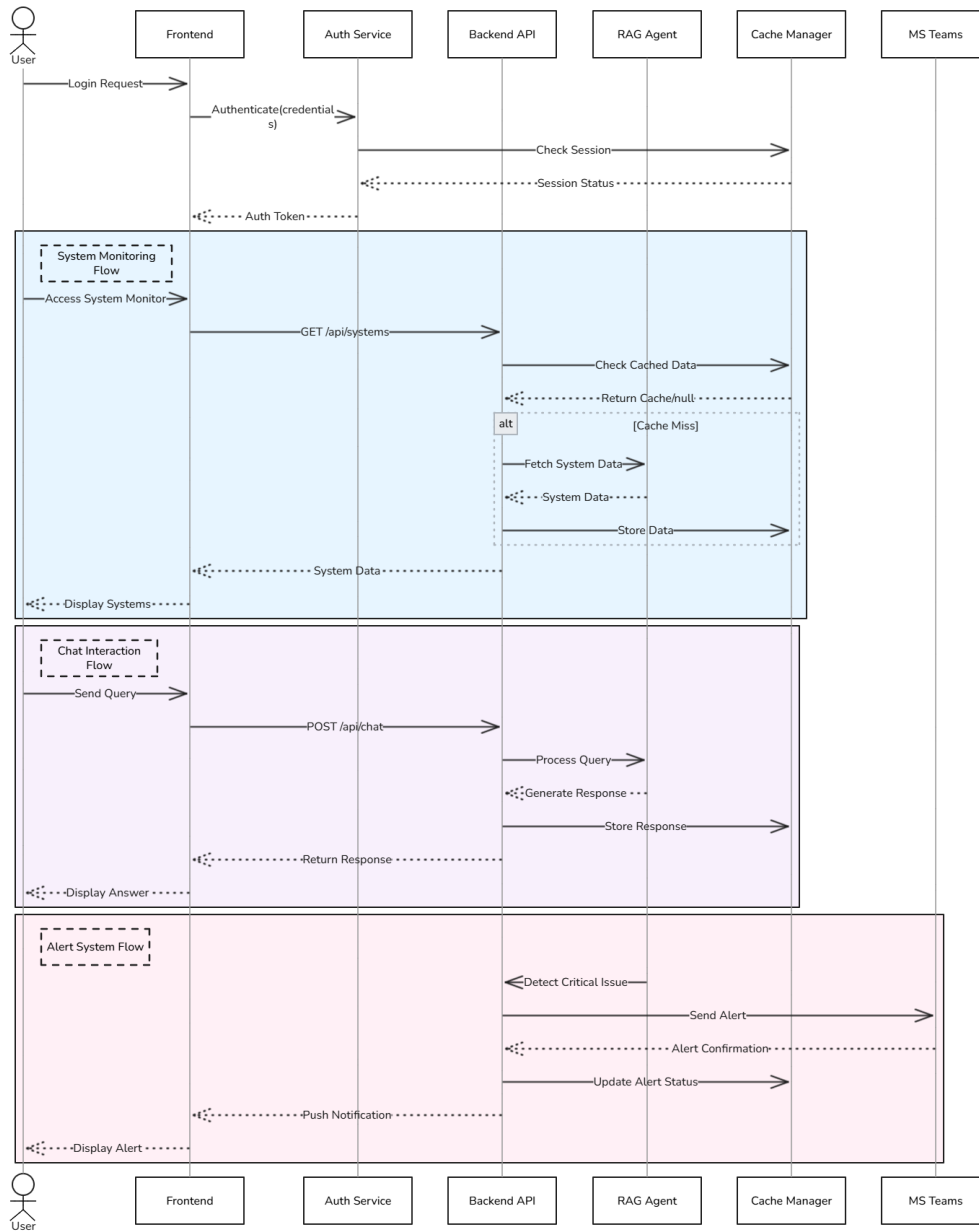


Figure 7.7: Complete System Interaction Flow - Authentication, Monitoring, Chat, and Alert Sequences

7.9.1 Use Case Diagram

Figure 7.8 summarizes key interactions between users and the system. It highlights core functionalities such as authentication, query handling, chatbot interactivity, T-code monitoring, and profile management.

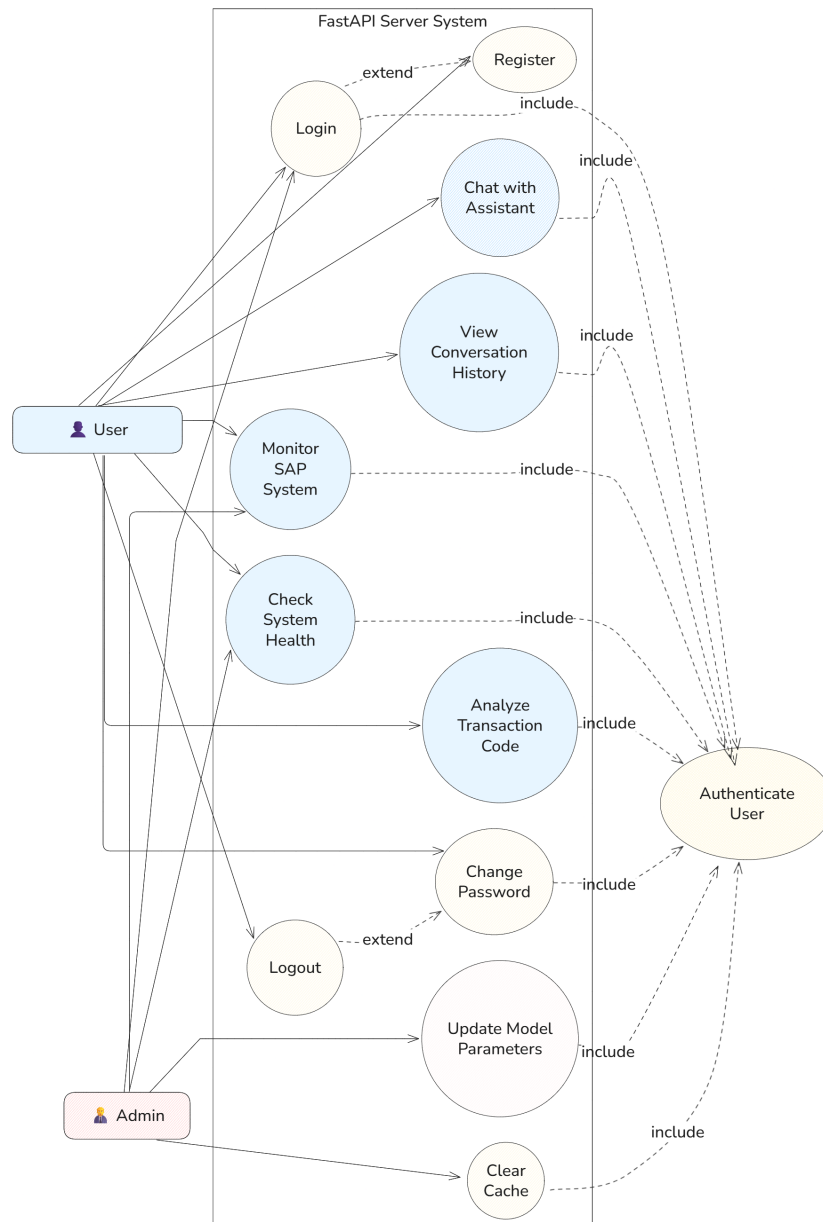
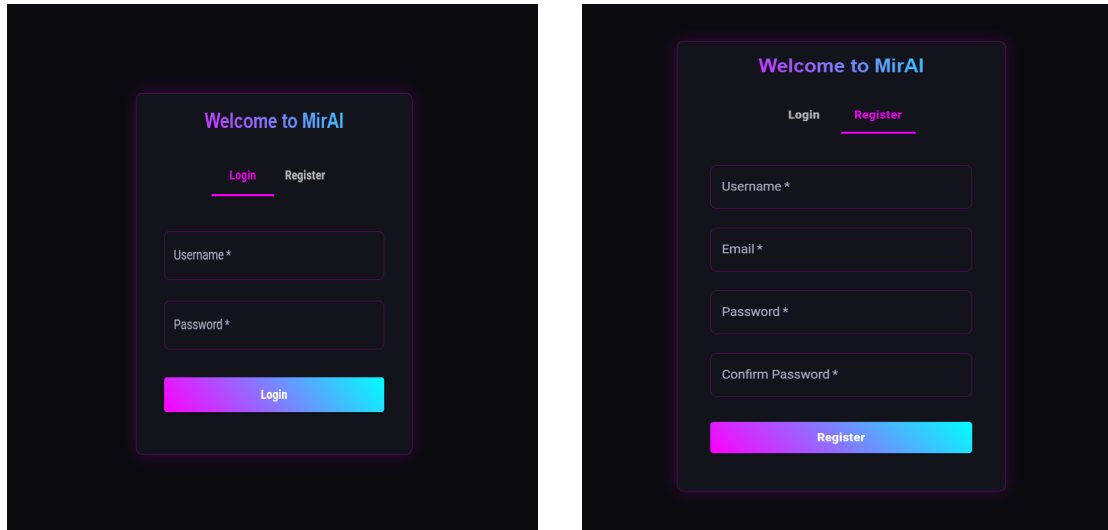


Figure 7.8: System Use Case Diagram

7.9.2 Authentication and Access Control

The system implements a secure authentication flow using JWT tokens with refresh token capabilities:



(a) Login interface for secure system access

(b) User registration interface for new account creation

Figure 7.9: View of login and registration interfaces

System Dashboard

The main dashboard provides an overview of all SAP systems with their current health status:

**Figure 7.10:** Systems Dashboard with Real-time Health Status

Transaction Code Monitoring

Each system can be monitored at both system-wide and transaction-specific levels:

Alert Generation and Visualization

The system generates two types of alerts:

System-wide Alerts

Alerts generated through continuous monitoring of system health:

The screenshot displays a user interface for system alerts. At the top, a dark purple header bar contains the text 'Analysis Details' with a downward arrow. Below this, a teal-colored section titled 'System Status Alert' with an upward arrow contains the following information:

ALERT: Potential Issues Identified in SAP System PI1 SEVERITY: Medium

DESCRIPTION: The SAP system PI1, running on Basis release 700 with Oracle as the database, has components that may be experiencing issues related to data handling and system integration. Specifically, Infotype 1061 Web Link errors and SLD data supplier configurations are flagged.

ROOT CAUSE:

1. Infotype 1061 Web Link exceeds the 100-character limit, likely causing data import/export issues.
2. The SLD data supplier may not be correctly extended, affecting system communication and data replication.

RECOMMENDATION:

1. Apply SAP [Note 1287958](#) to fix the Infotype 1061 issue.
2. Implement SAP [Note 1314774](#) to extend the SLD data supplier for SMSY.
3. Consider upgrading the Basis release to a newer version for better compatibility.
4. Monitor system performance using tools like ST03, ST06, and SM66 to identify bottlenecks.

RELATED TCODES:

- ST03 (Performance Monitoring)
- ST06 (Database Performance)
- SM66 (Background Job Monitoring)

SAP NOTES:

- 1287958
- 1314774

By addressing these issues, the system's stability and performance can be improved, reducing the risk of business operation disruptions.

Below the alert details, a 'Context Details' section is visible, showing a total of 50 items. It includes three expandable categories:

- Vector Context** (20 items)
- Graph Context** (10 items)
- Transaction Context** (Top 3 items)

Figure 7.11: system-wide alert with traceability context

Key components:

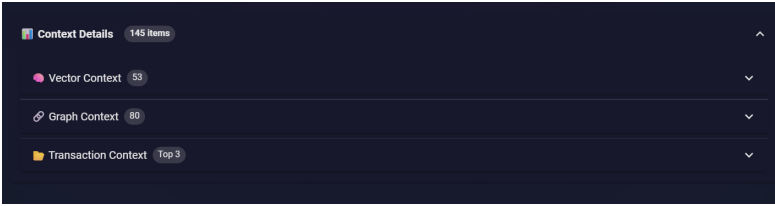
- Performance metrics visualization
- System health indicators
- Critical thresholds monitoring
- Historical trend analysis

Transaction-specific Alerts

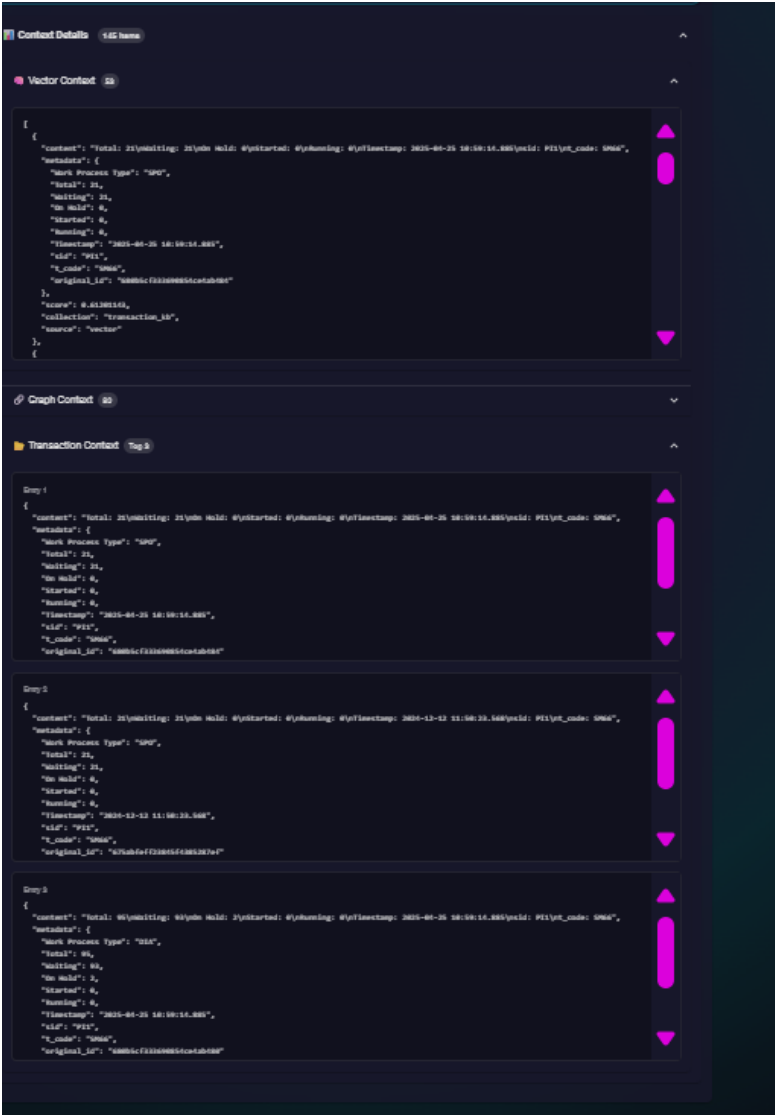
Detailed monitoring for specific transaction codes:

The screenshot displays the SAP Transaction Code Monitoring interface. At the top, it shows 'P11 (Active)' and 'SID: P11 | Type: SAP | Transactions: 1107'. A dropdown menu for 'Select Transaction Code' is set to 'SM66'. Below this, a row of transaction code buttons is shown, with 'SM66' highlighted in pink. The main section is titled 'Monitoring Result for SM66'. Under the 'Analysis Details' tab, a 'System Status Alert' is displayed. The alert text reads: 'ALERT: High Number of Waiting Work Processes in Transaction SM66', 'SEVERITY: Medium', 'DESCRIPTION: The transaction data shows that all work processes associated with SM66 are in a waiting state (e.g., 21 out of 21 on 2025-04-25 and 93 out of 95 on the same date). This indicates a potential bottleneck or issue preventing work processes from running.', 'ROOT CAUSE: The issue could be related to a known problem described in SAP Note 1278913, which involves dumps caused by many index partitions in the SAP_COLLECTOR_FOR_PERFMONITOR. Additionally, SAP Note 1293587 suggests issues with job deletions across different clients, which might also be contributing to the problem.', 'RECOMMENDATION: 1. Check the system logs for any dumps related to the SAP_COLLECTOR_FOR_PERFMONITOR. 2. Apply the corrections from SAP Note 1278913 to address the dump issue. 3. Investigate job scheduling and client-specific configurations to prevent unintended job deletions as per SAP Note 1293587. 4. Monitor work process distribution and adjust parameters if necessary to ensure optimal system performance.', 'RELATED TCODES: SM66, SM37, ST03, ST06', 'SAP NOTES: 1278913: Addresses a dump issue in the SAP_COLLECTOR_FOR_PERFMONITOR caused by many index partitions. 1293587: Fixes an issue where SUB jobs from different clients are deleted, which could impact job scheduling and system performance.', 'Summary: The high number of waiting work processes in SM66 suggests underlying issues that could be related to performance monitoring dumps or job scheduling problems. Applying the relevant SAP notes and investigating system configurations will help resolve these issues and improve system performance.' At the bottom, there is a 'Context Details' section with '145 items'.

Figure 7.12: Transaction Code Monitoring Output Example



(a) display of context details



(b) T-Code Monitoring Results

Figure 7.13: Transaction Code Context from databases

Alert components include:

- Response time analysis
- Error rate monitoring
- Transaction volume tracking
- Related SAP notes and documentation

7.9.3 Alert Notification System

To alert the BASIS monitoring team, the system sends real-time notifications to an MS Teams channel.

The MS Teams bot listens for webhook triggers from the RAG system when an anomaly is detected. Once data is processed and anomalies are identified (e.g., a critical runtime error for a specific `t_code` or an unexpected job status), it triggers the Incoming Webhook connector to the MS Teams channel.

Adaptive Card Implementation

The Teams integration uses Adaptive Cards for rich, interactive notifications:

Listing 7.4: Teams Card Creation

```
def create_teams_card(system_id: str,
                      metrics: Dict[str, Any],
                      analysis: str) -> Dict[str, Any]:
    severity, color, hex_color = determine_alert_severity(metrics)
    metrics_summary = format_metrics_summary(metrics)
    # Card structure implementation
```

- Alerts are triggered by abnormal transaction patterns or just the presence of a critical transaction pre-defined in the KB.
- Each alert contains transaction metadata and recommended actions
- Alerts link back to the UI for traceability and debugging

Incoming Webhooks in MS Teams

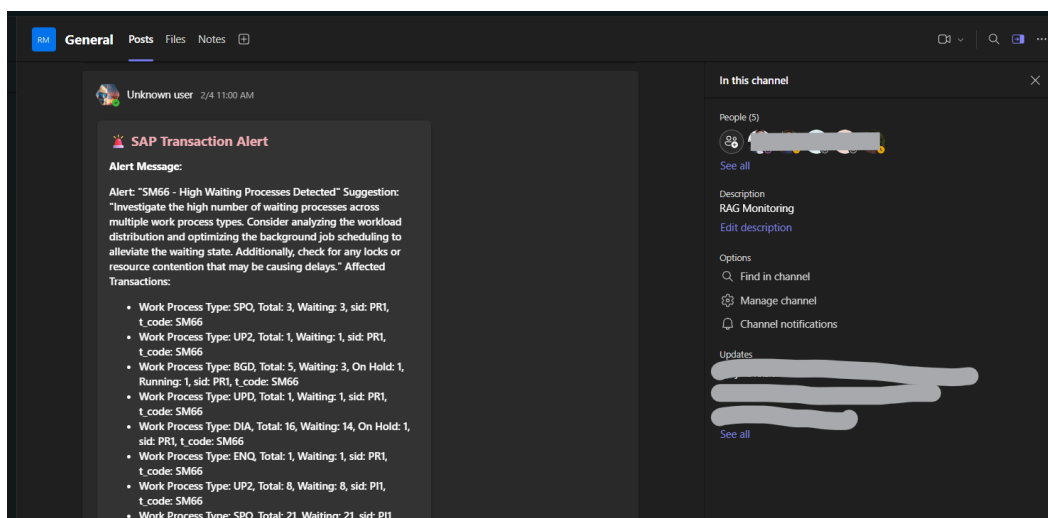


Figure 7.14: Example of MS Teams Notification for SAP Anomalies

MS Teams supports the use of **Incoming Webhooks** for third-party applications to post messages to a channel. The RAG system leverages this feature to send notifications about anomalies directly to a Teams channel. Here's how it works:

1. **Webhook URL Creation:** An incoming webhook is configured within a specific MS Teams channel.

2. **Payload Structure:** the RAG system writes the alert information into a JSON payload.

3. **Message Formatting:** The RAG system customizes the message using Markdown, ensuring that the notifications are clear and readable. The message typically includes the following information:

- **T_Code:** The SAP transaction code involved.
- **Error Type:** A description of the error or issue (e.g., runtime error).
- **Job ID:** If relevant, the unique identifier for the job involved.
- **Time window:** When the issue occurred (can be date , time , start date , end date, etc.).
- **Suggested Action:** Recommendations for resolving the issue, such as checking logs or reviewing configurations.
- **Associated Transactions:** Includes a portion of the group of transactions that raised the alert.

7.9.4 Interactive Chatbot Interface

Technical Architecture

The chatbot implementation consists of several key components:

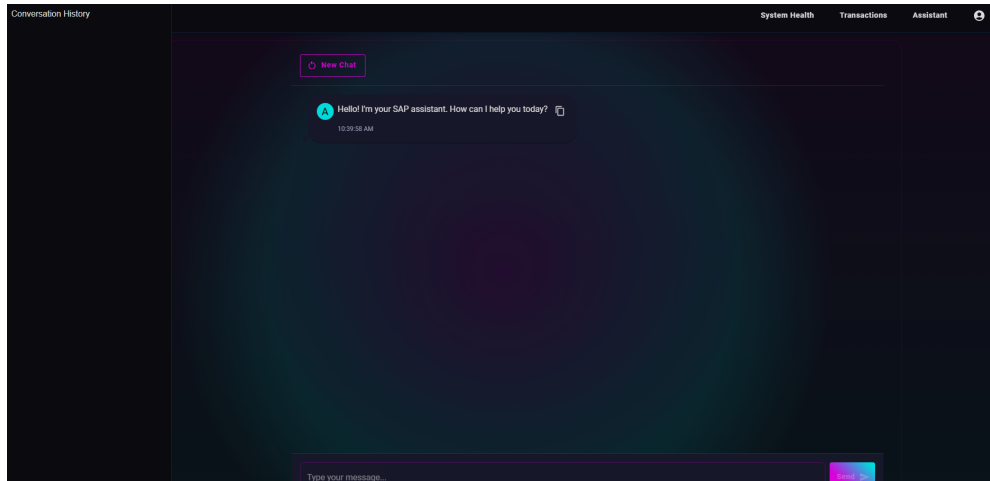
- **Frontend Interface:** React-based UI with Material-UI components
- **Backend API:** FastAPI server handling requests and RAG operations
- **Vector Store:** Qdrant for semantic search capabilities
- **Graph Database:** Neo4j for relationship-based context
- **LLM Integration:** Groq for response generation

7.9.5 Core Features Implementation

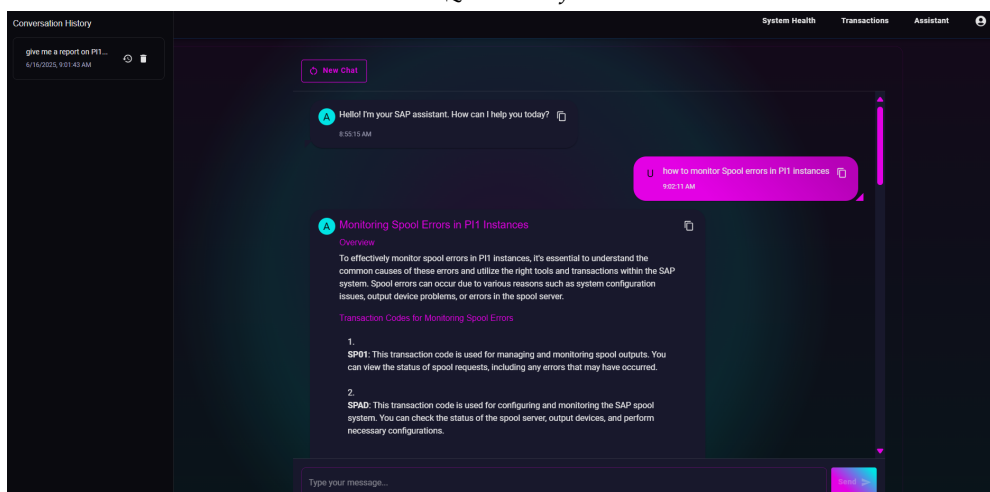
Message History Management

The chatbot maintains conversation context through:

- Persistent storage of chat sessions in Qdrant
- Context window management for relevant history
- Session-based memory for continuous conversations



(a) Q&A Interface



(b) Context-aware Responses

Figure 7.15: Chatbot Interaction Examples

Response Generation Pipeline

Messages flow through a sophisticated processing pipeline:

1. Query analysis and context extraction
2. Parallel retrieval from vector and graph stores
3. Context merging and prioritization
4. LLM-based response generation with templates
5. Response validation and formatting

7.9.6 User Interface Components

The interface features several key elements:

- Message bubbles with markdown support
- Conversation history panel
- Real-time typing indicators

- Copy and save functionality
- System context awareness indicators

7.9.7 Performance Optimization

Several optimizations ensure responsive performance:

- Response caching for common queries
- Batch processing for context retrieval
- Lazy loading of conversation history
- Efficient context window management
- Background processing for non-critical operations

*All inference models used in this pipeline, with the exception of cloud-hosted LLM, were executed locally with **CUDA acceleration** and FP16 (half-precision floating point) computation. This approach significantly reduces memory consumption and increases inference throughput on the GPU, enabling ideal performance even with resource-intensive transformer architectures.*

User Profile and Model Configuration Interface

The deployment also includes a user profile interface that allows authenticated users to update their account's password and view the configuration of the language models used in the application (Figure 7.16). This interface provides transparency for end-users by providing these features:

- **User Information Display:** Shows the username and email address of the logged-in user.
- **Password Management:** Secure fields are provided to update the password after verifying the current one.
- **Model Configuration Overview:** Displays the settings of the current language models, including temperature, token limits, and request timeouts.

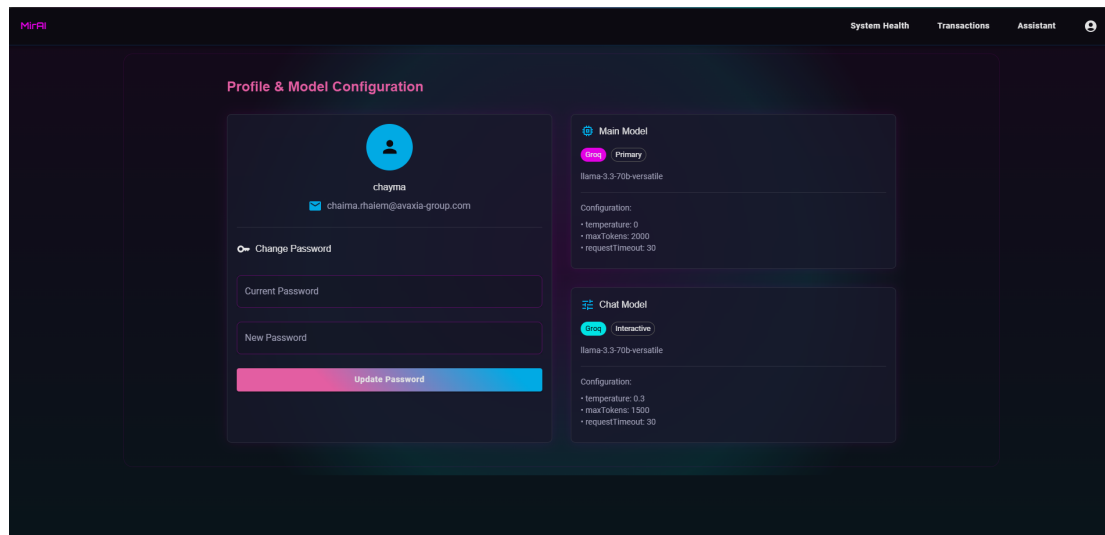


Figure 7.16: User Profile and Model Configuration Page

Password Update API

The password change functionality is securely handled through two backend endpoints:

1. **Password Validation** The endpoint `/api/user/validate-password`. If the password is incorrect or missing, it responds with a relevant HTTP error.

Listing 7.5: Password Validation Endpoint

```
@app.post("/api/user/validate-password")
async def validate_password(...):
    ...
```

2. **Password Update** After validation, users can submit their new password via the `/api/user/update-password` endpoint. The password is then hashed the new password and updated.

Each model's configuration is displayed with these parameters:

- temperature
- maxTokens
- requestTimeout

These values can only be updated from the backend by administrators to adjust model behavior without redeploying the frontend.

Conclusion

This modular and hybrid deployment setup allowed for a smooth integration of the project's components. It works for both local and cloud deployment. Additionally, it combines a modern UI, APIs, and messaging into one unified pipeline.

CONCLUSION AND PERSPECTIVES

This graduation project set out to design and implement an innovative prototype monitoring and alerting system tailored to the specific needs of SAP Basis environments. By leveraging the synergy of Graph-enhanced Retrieval-Augmented Generation (RAG) architectures, large language models (LLMs), and knowledge graph technologies, we were able to detect anomalies in SAP transaction codes, provide contextual explanations, and help in augmenting human decision-making through real-time alerts and chatbot interactions.

Throughout the project, considerable effort went into constructing a robust data pipeline, extracting and organizing SAP documentation, constructing a dense vector-based and graph-based knowledge base, and deploying a complete assistant with backend logic, UI interface, and Microsoft Teams integration. The evaluation methods, including G-Eval for graph triplet quality and precision analysis for vector retrieval, demonstrate promising performance in anomaly detection, semantic alerting, and end-user explainability.

At its core, this project demonstrates a powerful concept: intelligence can occasionally arise from structure, integration, and deliberate design rather than always from scale. This work demonstrates that well-designed hybrid systems, rooted in human context and informed by knowledge, can generate useful value where it really counts, on the front lines of enterprise operations, in a world that is constantly striving for ever-larger models.

Perspectives and Future Work

Even though the current system is a solid starting point, there are still a number of ways to improve the platform's resilience, scalability, and enterprise-readiness:

- **Reinforcement Learning from Human Feedback (RLHF):** Integrating RLHF mechanisms to adjust the chatbot's responses and the tone of alerts it generates based on user feedback is one significant area for improvement. This would improve the outputs' relevance, tone sensitivity, and actionability by allowing for ongoing learning from BASIS consultants and system administrators.

-
- **Integration of SAP Fiori/UI5 Frontends:** While a web-based and Teams-integrated interface has been developed, future versions of the assistant can explore embedding capabilities directly into the SAP Fiori ecosystem. This would allow seamless workflows for consultants within their native SAP GUI and enhance usability for enterprise clients.
 - **Security and Role-Based Access Control (RBAC):** As the solution moves closer to production-level deployment, reinforcing the authentication and authorization pipeline with enterprise-grade RBAC mechanisms and OAuth2 protocols will be critical to ensure compliance with corporate security policies.
 - **Scalability to Larger SAP Landscapes:** The current solution was designed and tested on selected t_codes and knowledge corpora. Scaling the knowledge base to accommodate full SAP landscapes, including industry-specific modules (e.g., SAP IS-U), requires enhanced document ingestion, better chunking strategies, and possibly horizontal vector store scaling.
 - **Multilingual and Cross-System Reasoning:** Future iterations could explore multilingual support (for global SAP deployments) and enhance the assistant's ability to perform reasoning across multiple systems and interdependent SAP modules using extended cross-graph traversal and hybrid retrieval strategies.
 - **Advanced Anomaly Detection Techniques:** While the current architecture uses hybrid semantic detection strategies, integrating statistical or ML-based anomaly detectors (e.g., Isolation Forests, LSTM-based forecasting) can improve sensitivity to abnormal behavior over time.
 - **Time-Series Anomaly Detection Models:** Rule and embedding-based anomaly detection is dominant in existing anomaly detection. Incorporating specially crafted models for anomaly detection in time-series (e.g., Prophet, LSTM-based models) can enhance early-warning and uncover temporal transaction patterns.
 - **Real-Time Change Data Capture (CDC):** Lightweight CDC pipelines would facilitate close to real-time ingestion of transactional updates with very little latency between transaction anomaly occurrence and alert generation.

In conclusion, this project demonstrates the concrete promise of connecting structured knowledge representation with sophisticated generative AI capabilities for augmenting SAP monitoring methods. By anchoring alerts in knowledge graphs and facilitating semantic reasoning via RAG, we take a step toward constructing explainable, context-aware enterprise assistants. With ongoing refinement and feedback-driven adjustment, this system can mature into an in-depth digital co-pilot for SAP administrators.

REFERENCES

1. AVAXIA Official Website: <https://www.avaxiagroup.com/> (Last accessed: October 24, 2024)
2. IBM SPSS Modeler Documentation – CRISP-DM Overview: <https://www.ibm.com/docs/fr/spss-modeler/18.5.0?topic=dm-crisp-help-overview> (Last accessed: October 25, 2024)
3. Knowledge Discovery and Data Mining: <https://www.geeksforgeeks.org/dbms/kdd-process-in-data-mining/> (Last accessed: October 25, 2024)
4. Microsoft TDSP Documentation: <https://learn.microsoft.com/en-us/azure/architecture/data-science-process/overview> (Last accessed: November 2, 2024)
5. Hugging Face Official Website: <https://huggingface.co/> (Last accessed: March 18, 2025)
6. Weaviate Blog – Introduction to Retrieval-Augmented Generation (RAG): <https://weaviate.io/blog/introduction-to-rag> (Last accessed: January 7, 2025)
7. Meta AI Blog – LLaMA Model Introduction: <https://ai.meta.com/blog/large-language-model-llama-meta-ai/> (Last accessed: March 25, 2025)
8. PyGeometric GitHub Repository: https://github.com/pyg-team/pytorch_geometric (Last accessed: February 10, 2025)
9. Visual Studio Code: <https://code.visualstudio.com/> (Last accessed: November 15, 2024)
10. VMware Horizon Client Portal: <https://cloud.vmwarehorizon.com/appblast/webclient/> (Last accessed: March 4, 2025)
11. Uvicorn Official Website: <https://uvicorn.org/> (Last accessed: April 10, 2025)
12. FastAPI Documentation: <https://fastapi.tiangolo.com/> (Last accessed: April 12, 2025)
13. TypeScript Documentation: <https://www.typescriptlang.org/> (Last accessed: April 17, 2025)
14. React Official Website: <https://react.dev/> (Last accessed: April 17, 2025)
15. Ollama Official Website: <https://ollama.com/> (Last accessed: April 22, 2025)
16. Groq Official Website: <https://groq.com/> (Last accessed: April 23, 2025)
17. Vaswani et al., "Attention is All You Need", 2017 (Transformer paper): <https://arxiv.org/abs/1706.03762> (Last accessed: February 15, 2025)
18. Devoteam Blog – Generative AI Hierarchy:
<https://www.devoteam.com/expert-view/unlimited-creativity-how-generative-ai-is-transforming-the-world-of-innovation/>
(Last accessed: March 30, 2025)
19. Gartner Hype Cycle for Artificial Intelligence 2024: <https://www.gartner.com/en/articles/hype-cycle-for-artificial-intelligence> (Last accessed: October 28, 2024)

BIBLIOGRAPHY

- Bordes, Antoine et al. (2013). “Translating embeddings for modeling multi-relational data”. In: *Advances in neural information processing systems*. Accessed February 5, 2025, pp. 2787–2795.
- Grover, Aditya and Jure Leskovec (2016). “node2vec: Scalable feature learning for networks”. In: *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. Accessed February 18, 2025, pp. 855–864.
- He, Xiaoxin et al. (2024). “G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv preprint arXiv:2402.07630 , Accessed April 5, 2025.
- Lin, Yankai et al. (2015). “Learning entity and relation embeddings for knowledge graph completion”. In: *AAAI Conference on Artificial Intelligence*. Vol. 29. 1. Accessed February 18, 2025.
- Mialon, Gaël et al. (2023). “Retrieval-Augmented Generation for Large Language Models: A Survey”. In: *arXiv preprint arXiv:2301.00375*. Accessed March 29, 2025.
- Perozzi, Bryan, Rami Al-Rfou, and Steven Skiena (2014). “DeepWalk: Online learning of social representations”. In: *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. Accessed February 18, 2025, pp. 701–710.
- Sun, Zhiqing et al. (2019). “RotatE: Knowledge graph embedding by relational rotation in complex space”. In: *International Conference on Learning Representations (ICLR)*. Accessed February 18, 2025.
- Trouillon, Théo et al. (2016). “Complex embeddings for simple link prediction”. In: *International Conference on Machine Learning (ICML)*. Accessed February 18, 2025, pp. 2071–2080.
- Vaswani, Ashish and et al. (2017). “Attention is all you need”. In: *Advances in Neural Information Processing Systems*. Accessed November 7, 2024.
- Wang, Hanlin et al. (2024). “HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction”. In: *arXiv preprint arXiv:2408.04948*. Accessed April 19, 2025.
- Yang, Bishan et al. (2015). “Embedding entities and relations for learning and inference in knowledge bases”. In: *International Conference on Learning Representations (ICLR)*. Accessed February 18, 2025.



ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454